PLATFORM ARCHITECTURE

# Brand-Field Platform Plan

Multi-Tenant Brand Architecture for AI Salon, Coma, and Future Client Brands

Document version: 1.0

Owner: MassaPro Platform Agent

Date: August 2026

Classification: Internal — Engineering & Product

# Table of Contents

*Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."*

# 1. Executive Summary

The AI Salon platform currently operates as a single-brand application: the string "AI Salon" is hardcoded across 185 source files (565 occurrences), the production host aisalon.massapro.com is the only supported domain, the login page defaults to the Tel Aviv chapter, and email templates embed the brand name, signature, and logo URL directly in their HTML. A partial brand-image abstraction exists via SiteSetting and ChapterSetting for four image keys (favicon, login hero, login banner, email logo), but no Brand, Tenant, or Whitelabel model exists in the Prisma schema, and no host-based dispatch logic exists in the middleware (which today only performs UTM referral tracking).

This document specifies the refactor that converts the platform into a **multi-tenant brand platform** serving two brands immediately — **Coma** (the parent company, accessible at coma.massapro.com) and **AI Salon** (the global chapters brand, accessible at aisalon.massapro.com) — and N future client brands with zero additional code. The architecture introduces a new Brand model in Prisma, host-based brand resolution in Next.js middleware, a single getEffectiveBrand(context) resolver that all consumers call, and a brand-tokenized email renderer that supports the Coma-above-chapter logo coexistence rule (the parent Coma mark is always rendered above the chapter brand mark, both on web pages and in email headers and signatures). Existing users see no change — the AI Salon brand loads as today. New users registering via coma.massapro.com/login see the Coma brand and are tagged with brandId = 'coma' on signup.

The governance model for this refactor is explicit: **the platform — schema, middleware, brand resolver, email renderer, admin Brand Management UI, migrations, and the file-by-file change inventory — is owned and managed in this agent thread**. A parallel agent thread is responsible **only** for the login page UI/UX implementation under the brand configuration produced here. The boundary between the two agents is the Brand model and the getEffectiveBrand() resolver: this agent defines both, the parallel agent consumes them. New brand additions (future client brands) are performed by this agent via the admin UI and require no parallel-agent involvement, because the login page is brand-agnostic by design.

The implementation is scoped as a **single sprint (~1–2 weeks)** broken into nine PRs, with the platform agent owning eight and the parallel agent owning one (the login page brand-swap). A feature flag BRAND_DOMAIN_RESOLVE_ENABLED gates the host-based dispatch, enabling zero-downtime rollout and instant rollback to the current single-brand behavior. Key risks center on the 150-file metadata refactor (mitigated by an automated codemod), middleware host-detection edge cases on Vercel (mitigated by the feature flag and a fallback to the AI Salon default), and the brand-perception impact of the Coma parent logo on AI Salon chapter emails (mitigated by a per-

brand parentLogoPosition setting that allows the parent mark to be moved to the footer or hidden entirely for specific chapters).

## 2. Background, Goals & Governance Model

### 2.1 Current State

The AI Salon platform is a Next.js 16 App Router application backed by Prisma + PostgreSQL and nodemailer SMTP. The Prisma schema contains 44 models, organized around the hierarchy Country → Chapter → User, with admin roles encoded on the User.role enum (SUPER_ADMIN, ADMIN, CHAPTER_ORGANIZER, CO_HOST, SPEAKER, MEMBER). There is **no Brand, Tenant, Organization, or Whitelabel model** anywhere in the schema. The closest existing abstractions to a tenant are the Country and Chapter tables, and the only brand-field storage is image-only: four keys (favicon, loginHero, loginBanner, emailLogo) stored in SiteSetting (global) and ChapterSetting (per-chapter override).

A codebase-wide grep for the patterns AI Salon | AISalon | ai-salon | aisalon returns 565 occurrences across 185 files in /src, plus 3 files in /prisma and 8 in /public. The largest concentrations are in the email subsystem (src/lib/email.ts with 28 occurrences, src/lib/email-orchestrator/templates.ts with 18), admin UI strings (src/app/admin/knowledge-base/page.tsx with 14), the login page (13), the chapter onboarding flow (22 across two files), and approximately 150 admin/page.tsx files each exporting a metadata = { title: "X — AI Salon" } object. The production host aisalon.massapro.com is also hardcoded in next.config.ts images.remotePatterns, in the email SHELL footer URL, and in the SMTP_FROM env-var example.

The middleware (src/middleware.ts, 227 lines, Node runtime) currently does **only UTM referral tracking** — it sets a cookie, records a ReferralVisit row, and skips API/auth paths. There is no req.headers.host lookup, no x-forwarded-host handling, no per-domain routing, and no auth gate. The login page (src/app/login/page.tsx) hardcodes DEFAULT_CHAPTER_SLUG = "tel-aviv", uses the AiSalonLogoServer component, and embeds the AI Salon color palette (#FF005A, #00E6FF, #004F98, #820A7D) directly in the JSX. Multi-tenant behavior today is via ?chapterSlug= query parameter only.

Email sending has two parallel code paths. Transactional emails (src/lib/email.ts, 528 lines) use nodemailer SMTP with four builder functions (sendPasswordEmail, sendRsvpConfirmationEmail, sendChapterOnboardingEmail, sendChapterProvisionedEmail), each with hardcoded "AI Salon" signatures and a hardcoded fallback from-address "AI Salon Chat <chat@aisalon.massapro.com>". Orchestrator and campaign emails (src/lib/email-orchestrator/templates.ts + src/lib/email-campaign/render.ts) flow through a unified renderer (render-unified.ts) that replaces {{tokens}} and injects a logo block,

but the SHELL HTML wrapper hardcodes the lowercase wordmark "aisalon", the footer "AI Salon {{chapter_name}} · Empowering AI Connections", and the URL https://aisalon.massapro.com. An inconsistency exists today: transactional signatures link to massapro.com while the orchestrator SHELL footer links to aisalon.massapro.com — this refactor resolves both to the brand's canonical URL.

## 2.2 Target State

The target state is **one platform serving two brands now and N future client brands later**, with host-based dispatch as the only brand-selection mechanism at the edge. The two initial brands are:

- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]

Future client brands are added by a SUPER_ADMIN via the new /admin/brand screen, with zero code change. Each new brand declares its domain, brand fields (name, colors, logo URLs, from-address), and optionally a parent brand (typically Coma, but the architecture permits any parent-child relationship). The platform treats all brands uniformly: the resolver returns a BrandContext object containing the resolved brand and (if set) the parent brand, and every consumer reads from that object.

The login page is **brand-agnostic by design**: a single Next.js route (/login) reads the resolved brand from the request and swaps the logo, colors, strings, mascot, and chapter-slug default. There is no forked route, no separate codebase, no separate deployment. The parallel agent's only deliverable is the UI/UX polish on this single route — colors, hero copy, mascot illustration, gradient — under the brand fields this agent defines.

## 2.3 Governance Model — Platform vs. Login Agent

This refactor introduces an explicit governance boundary that must survive across sprints and future brand additions:

**Platform agent (this thread) owns:**

- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]

- [object Object],[object Object],[object Object],[object Object]
- [object Object]

**Login agent (parallel thread) owns:**

- [object Object],[object Object],[object Object],[object Object]
- [object Object]
- [object Object],[object Object],[object Object]

**Boundary contract:**

The platform agent exposes two interfaces to the login agent: (1) the `Brand` TypeScript type (exported from `src/lib/brand/types.ts`), and (2) a `getBrandForRequest()` async helper (from `src/lib/brand/resolver.ts`) that the login page calls at the top of its server component to receive a fully-resolved `BrandContext` object. The login agent then renders any visual treatment it wants using the brand fields. If the login agent needs a new brand field (e.g., a custom mascot URL), the request comes back to this agent, the field is added to the schema, and the login agent picks it up — the login agent never edits the schema.

## 2.5 Priority #1 — Login URL Contract & Domain Ownership Split

This section is the FIRST change to ship. It defines: (a) the canonical login URL contract that all brands must follow, (b) the ownership boundary between this platform agent and the parallel Coma-landing agent on `coma.massapro.com`, and (c) the routing rule that lets the platform own `/login` without touching the rest of the `coma.massapro.com` site. Everything else in this plan (Brand table, host resolver, email tokens, admin UI) builds on top of this contract. The login agent cannot begin implementation until §2.5.1 through §2.5.4 are agreed and the routing rule in §2.5.4 is provisioned.

### 2.5.1 Canonical Login URL Contract

Every login URL across every brand follows ONE shape. The `brand` query param is optional on the AI Salon host (defaults to `aisalon`) and on the Coma host (defaults to `coma`), but it can be passed explicitly to override the host default — this is what enables a single login route to render any external client brand (Google, a future customer, etc.) without code change. The `chapterSlug` param is also optional and falls back to the brand's `defaultChapterSlug` field.

**Table 0: Canonical login URL contract**

| URL | brand param | chapterSlug param | Resolves to brand | Resolves to chapter |
|---|---|---|---|---|
| https://coma.massapro.com/login | (none) | (none) | coma (host default) | null — Coma has no chapter context |
| https://coma.massapro.com/login?chapterSlug=tlv | (none) | tlv | coma (host default) | Tel Aviv (under coma brand context) |
| https://coma.massapro.com/login?brand=google&chapterSlug=tlv | google | tlv | google (URL override) | Tel Aviv — rendered with Google logo + Coma parent above |
| https://coma.massapro.com/login?brand=aisalon&chapterSlug=mtl | aisalon | mtl | aisalon (URL override) | Montreal — rendered with AI Salon logo + Coma parent above |
| https://aisalon.massapro.com/login | (none) | (none) | aisalon (host default) | Tel Aviv (brand defaultChapterSlug) |
| https://aisalon.massapro.com/login?chapterSlug=mtl | (none) | mtl | aisalon (host default) | Montreal |
| https://aisalon.massapro.com/login?brand=coma | coma | (none) | coma (URL override on AIS host) | null — Coma has no chapter context |

## 2.5.2 Brand Resolution Priority (4-tier)

The login page resolves the brand using a strict 4-tier precedence chain. The first non-empty tier wins; lower tiers are not consulted. This chain is implemented in `getBrandForRequest()` (defined in §3.4) and is the single source of truth for every server component, route handler, and middleware invocation. The URL param wins over the host so that demo URLs like `?brand=google` can be shared without DNS changes — this is critical for sales demos and partner previews.

[object Object],[object Object],[object Object],[object Object]

[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]

[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]

[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]

### 2.5.3 Domain Ownership Boundary (Platform vs Parallel Coma Agent)

The coma.massapro.com hostname is shared between two independent agents. This platform agent owns the login flow and the user-account database; the parallel Coma agent owns the marketing landing page (the root / and any non-/login path). The two agents must NOT modify each other's code or content. The boundary is enforced at the edge (Caddy / Vercel routing layer), not in the application code — the application has no awareness of the split.

**Table 0b: Path ownership on coma.massapro.com**

| Path | Owned by | Served by | Notes |
|---|---|---|---|
| coma.massapro.com/ | Parallel Coma agent | External service (e.g., separate Vercel project, static host, or CMS) | Marketing landing page. This agent MUST NOT touch. |
| coma.massapro.com/login | This platform agent | Next.js app (aisalon-massapro project) | Login form, brand resolver, signup. Owned here. |
| coma.massapro.com/login/* | This platform agent | Next.js app | Subpaths (forgot-password, set-password, etc.) — also owned here. |
| coma.massapro.com/api/* | This platform agent | Next.js app | All API routes for auth, signup, brand assets. Owned here. |
| coma.massapro.com/_next/* | This platform agent | Next.js app (static assets) | Next.js build output. Owned here. |
| coma.massapro.com/brand-assets/* | This platform agent | Next.js app (public/) | Brand image uploads. Owned here. |
| coma.massapro.com/<anything-else> | Parallel Coma agent | External service | Everything else (/, /about, /pricing, /blog, etc.) — Coma agent owns. |

### 2.5.4 Routing Rule — Edge-Level Path Split (NO Application Awareness)

The split is implemented at the edge (Caddy reverse proxy in this workspace, or Vercel's routing config in production). The Next.js application has zero awareness of the split — it serves routes as if it were the only app on the domain. The edge inspects the path prefix and routes to the right backend.

**Local dev / workspace (Caddyfile):** Edit /home/z/my-project/Caddyfile to add path-prefix routing for the coma host. The existing Caddy on port :81 already proxies all traffic to localhost:3000; we add a matcher for non-/login paths on the coma host that proxies to the Coma landing service instead.

```
# Caddyfile — add path-split routing for coma.massapro.com
# (existing :81 block stays for default proxy; this is a new block)

# COMA_PARALLEL_UPSTREAM env var points to the parallel agent's service
# e.g. https://coma-landing.vercel.app or http://localhost:3001

comalookup:
{$COMA_PARALLEL_UPSTREAM:https://coma-landing-or-placeholder.vercel.app} {
   # default fallback if env var unset
}

:81 {
   @coma_host {
      host coma.massapro.com www.coma.massapro.com
   }
   @coma_login_path {
      host coma.massapro.com www.coma.massapro.com
      path /login /login/* /api/* /_next/* /brand-assets/* /favicon.ico /robots.txt
   }
   @coma_root_path {
      host coma.massapro.com www.coma.massapro.com
      not path /login /login/* /api/* /_next/* /brand-assets/* /favicon.ico /robots.txt
   }

   # /login and friends -> Next.js app (this platform)
   handle @coma_login_path {
      reverse_proxy localhost:3000 {
         header_up Host {host}
         header_up X-Forwarded-For {remote_host}
         header_up X-Forwarded-Proto {scheme}
         header_up X-Real-IP {remote_host}
      }
   }

   # Everything else on coma.massapro.com -> parallel Coma agent
   handle @coma_root_path {
      reverse_proxy {$COMA_PARALLEL_UPSTREAM:https://coma-landing.vercel.app} {
         header_up Host coma.massapro.com
         header_up X-Forwarded-For {remote_host}
         header_up X-Forwarded-Proto {scheme}
      }
   }
```

```
    # All other hosts (aisalon.massapro.com, etc.) -> Next.js app as today
    handle {
        reverse_proxy localhost:3000 {
            header_up Host {host}
            header_up X-Forwarded-For {remote_host}
            header_up X-Forwarded-Proto {scheme}
            header_up X-Real-IP {remote_host}
        }
    }
}
```

**Production (Vercel):** Vercel does not support per-path upstream proxying natively on a single domain. The production equivalent is to use a Vercel Edge Middleware that returns a `NextResponse.rewrite()` to an external URL for non-`/login` paths on `coma.massapro.com`. The external URL is the parallel Coma agent's hosting (could be another Vercel project, a Netlify site, or any HTTP endpoint). This middleware lives in THIS codebase at `src/middleware.ts` but it ONLY matches the coma host — see §4.1 for the matcher config. The parallel Coma agent never sees this middleware; they only see HTTP requests arriving at their upstream.

```
// src/middleware.ts — path-split for coma.massapro.com (Vercel production)
// COMA_PARALLEL_UPSTREAM env var = parallel agent's URL (e.g. https://coma-
landing.vercel.app)

const COMA_HOSTS = new Set(['coma.massapro.com', 'www.coma.massapro.com']);
const PLATFORM_PATHS = /^\/(login|api|_next|brand-assets)(\/|$)/;

export async function middleware(req: NextRequest) {
  const host = (req.headers.get('x-forwarded-host') || req.headers.get('host') ||
'').toLowerCase();

  // === Path split for coma.massapro.com ===
  if (COMA_HOSTS.has(host) && !PLATFORM_PATHS.test(req.nextUrl.pathname)) {
    const upstream = process.env.COMA_PARALLEL_UPSTREAM;
    if (upstream) {
      // Rewrite to external URL — Vercel fetches it transparently
      const url = new URL(req.nextUrl.pathname + req.nextUrl.search, upstream);
      return NextResponse.rewrite(url);
    }
    // If upstream not set, fall through to Next.js (404 for unknown paths)
  }

  // === Brand resolution (existing — see §4.1) ===
  if (process.env.BRAND_DOMAIN_RESOLVE_ENABLED === 'true') {
    // ... brand resolution logic from §4.1 ...
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/((?!_next/static|_next/image|favicon\.ico|robots\.txt|sitemap\.xml).*)"],
};
```

### 2.5.5 Implementation Steps for Priority #1 (Ship First)

These steps are executed by the platform agent (this thread). They are blocking for the login agent — the login agent cannot test the new URL contract until step 5 is deployed. Estimated effort: 1 day.

[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]

[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]

[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]

[object Object],[object Object],[object Object],[object Object]

[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]

[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]

[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]

### 2.5.6 What the Parallel Coma Agent Must Do (Not Our Concern)

The parallel Coma agent has zero code dependencies on this platform. They only need to ensure their upstream HTTP service accepts requests with the `Host: coma.massapro.com` header (which Vercel/Caddy sets automatically). They do not need to know about the Brand table, the login page, or any of the platform internals. Their only contract is: "any HTTP request that arrives at your upstream is for a non-/login path on coma.massapro.com — render your landing page and return 200."

If the parallel agent ever wants to link from the landing page to the login (e.g., a "Sign In" button), the link target is just `https://coma.massapro.com/login` — no special params needed. The brand resolver handles the rest. They can also link to a specific chapter's login via `https://coma.massapro.com/login?brand=aisalon&chapterSlug=tlv` if they want to deep-link into the AI Salon chapter flow from the Coma landing page (e.g., "AI Salon Tel Aviv — Sign In").

### 2.5.7 Acceptance Criteria for Priority #1

- [object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object]

# 3. Brand Data Model & Prisma Schema

## 3.1 Prisma Model Definition

A new Brand model is added to prisma/schema.prisma. The model is intentionally flat (no JSON columns) for query-ability and for the admin UI's per-field form. Color fields are stored as 6-digit hex strings (e.g., "#D4875A"); the gradient CSS is stored as a raw CSS string for direct injection into a style attribute. The parentLogoUrl and parentLogoPosition fields encode the Coma-above-chapter coexistence rule on the brand itself (not on the chapter), so all chapters under a brand inherit the same parent-mark behavior — but per-chapter overrides remain possible via the existing ChapterSetting table.

```
model Brand {
  id                String   @id @default(cuid())
  slug              String   @unique        // e.g. "aisalon", "coma"
  name              String                  // Display name: "AI Salon", "Coma"
  wordmark          String                  // Lowercase identifier: "aisalon", "coma"
  tagline           String?                 // "Empowering AI Connections"
  description       String?

  // Domain routing
  domain            String   @unique        // "aisalon.massapro.com"
  altDomains        String[]                // ["www.aisalon.massapro.com"]
  siteUrl           String                  // "https://aisalon.massapro.com"
  defaultChapterSlug  String?               // chapter to land on if none specified

  // Email
  supportEmail      String?                 // "support@aisalon.massapro.com"
  fromName          String                  // "AI Salon Chat"
  fromEmail         String                  // "chat@aisalon.massapro.com"
  replyTo           String?

  // Brand images
  logoUrl           String?                 // primary mark
  logoDarkUrl       String?                 // dark-bg variant
  faviconUrl        String?
  emailLogoUrl      String?                 // email header (150px wide)
  loginHeroUrl      String?                 // login page hero image
  loginBannerUrl    String?                 // login page banner mark

  // Colors
  primaryColor      String?                 // "#FF005A"
  secondaryColor    String?
  accentColor       String?
  gradientCss       String?                 // "conic-gradient(from 180deg, ...)"

  // Parent brand coexistence (null on Coma, set on AI Salon + future client brands)
  parentLogoUrl     String?                 //
"https://coma.massapro.com/assets/coma_trans.png"
  parentLogoPosition  ParentLogoPosition @default(ABOVE)
  parentBrandName   String?                 // "Coma" (denormalized for email rendering)
  parentBrandUrl    String?                 // "https://coma.massapro.com"
```

```
  isActive           Boolean  @default(true)
  sortOrder          Int      @default(0)
  createdAt          DateTime @default(now())
  updatedAt          DateTime @updatedAt

  // Relationships
  countries          Country[]
  users              User[]
  chapters           Chapter[]              // via Country (computed; see resolvers)

  @@map("brand")
}

enum ParentLogoPosition {
  ABOVE          // Coma logo above chapter logo (default for AI Salon)
  BESIDE         // Coma logo beside chapter logo
  FOOTER_ONLY    // Coma logo in footer only (web + email signature)
  HIDDEN         // No parent logo (used by Coma brand itself)
}
```

Three existing models receive a nullable `brandId` foreign key. The migration backfills every existing row with `brandId = 'aisalon'`, then (in a follow-up migration once verification passes) the column is made non-nullable. The `Chapter` relationship to `Brand` is computed via `Country` — chapters do not carry their own `brandId`, they inherit it from their country. This avoids denormalization and keeps the chapter-onboarding flow (which creates a `Chapter` from a `Country`) unchanged.

```
// Added to existing models:
model User {
  // ... existing fields ...
  brandId   String?   // → backfilled to "aisalon", then NOT NULL
  brand     Brand?    @relation(fields: [brandId], references: [id])
}

model Country {
  // ... existing fields ...
  brandId   String?
  brand     Brand?    @relation(fields: [brandId], references: [id])
}

model Chapter {
  // ... existing fields ...
  // brandId is NOT on Chapter — it's derived via Country.brand
}
```

## 3.2 Backfill Data — Two Initial Brands

Two `Brand` rows are inserted by the migration's backfill step. Coma is inserted first because AI Salon references it as a parent.

```
-- Step 1: Insert parent brand (Coma)
INSERT INTO brand (
```

```
  id, slug, name, wordmark, tagline, domain, site_url,
  from_name, from_email, support_email,
  logo_url, favicon_url, email_logo_url, login_hero_url, login_banner_url,
  primary_color, secondary_color, accent_color, gradient_css,
  parent_logo_position, is_active, sort_order, created_at, updated_at
) VALUES (
  'brand_coma', 'coma', 'Coma', 'coma', 'The platform behind AI Salon',
  'coma.massapro.com', 'https://coma.massapro.com',
  'Coma', 'no-reply@coma.massapro.com', 'support@coma.massapro.com',
  'https://coma.massapro.com/assets/coma_trans.png',
  'https://coma.massapro.com/assets/coma_favicon.png',
  'https://coma.massapro.com/assets/coma_trans.png',
  'https://coma.massapro.com/assets/coma_login_hero.png',
  'https://coma.massapro.com/assets/coma_login_banner.png',
  '#0F172A', '#475569', '#0042E6',
  'linear-gradient(135deg, #0F172A 0%, #475569 100%)',
  'HIDDEN', true, 0, NOW(), NOW()
);

-- Step 2: Insert child brand (AI Salon) — references Coma as parent
INSERT INTO brand (
  id, slug, name, wordmark, tagline, domain, site_url, default_chapter_slug,
  from_name, from_email, support_email,
  logo_url, favicon_url, email_logo_url, login_hero_url, login_banner_url,
  primary_color, secondary_color, accent_color, gradient_css,
  parent_logo_url, parent_logo_position, parent_brand_name, parent_brand_url,
  is_active, sort_order, created_at, updated_at
) VALUES (
  'brand_aisalon', 'aisalon', 'AI Salon', 'aisalon', 'Empowering AI Connections',
  'aisalon.massapro.com', 'https://aisalon.massapro.com', 'tel-aviv',
  'AI Salon Chat', 'chat@aisalon.massapro.com', 'support@aisalon.massapro.com',
  'https://aisalon.massapro.com/brand/aisalon-logo.webp',
  'https://aisalon.massapro.com/favicon.webp',
  'https://aisalon.massapro.com/brand/aisalon-logo.webp',
  'https://aisalon.massapro.com/images/falafel-meerkat.jpg',
  'https://aisalon.massapro.com/brand/aisalon-logo.webp',
  '#FF005A', '#00E6FF', '#820A7D',
  'conic-gradient(from 180deg at 50% 50%, #FF005A, #820A7D, #004F98, #00E6FF,
#FF005A)',
  'https://coma.massapro.com/assets/coma_trans.png', 'ABOVE', 'Coma',
'https://coma.massapro.com',
  true, 1, NOW(), NOW()
);

-- Step 3: Backfill FKs on existing rows
UPDATE "user"    SET brand_id = 'brand_aisalon' WHERE brand_id IS NULL;
UPDATE country   SET brand_id = 'brand_aisalon' WHERE brand_id IS NULL;
-- Chapter inherits via Country — no chapter.brand_id column.

-- Step 4: Insert example external client brand (google — for sales demos)
-- This row is what makes https://coma.massapro.com/login?
brand=google&chapterSlug=tlv
-- render the Google logo + Coma parent above + "Tel Aviv" as the chapter name.
INSERT INTO brand (
  id, slug, name, wordmark, tagline,
  domain, site_url, default_chapter_slug,
  from_name, from_email, support_email,
  logo_url, favicon_url, email_logo_url, login_hero_url, login_banner_url,
  primary_color, secondary_color, accent_color, gradient_css,
```

```
  parent_logo_url, parent_logo_position, parent_brand_name, parent_brand_url,
  is_active, sort_order
) VALUES (
  'brand_google', 'google', 'Google', 'google', 'Organize the world''s information',
  'google.com', 'https://www.google.com', NULL,
  'Google Workspace', 'no-reply@google.com', 'support@google.com',

'https://www.google.com/images/branding/googlelogo/2x/googlelogo_color_272x88dp.png',
  'https://www.google.com/favicon.ico',

'https://www.google.com/images/branding/googlelogo/2x/googlelogo_color_272x88dp.png',
  NULL, NULL,
  '#4285F4', '#34A853', '#EA4335',
  'linear-gradient(135deg, #4285F4 0%, #34A853 100%)',
  'https://coma.massapro.com/assets/coma_trans.png', 'ABOVE', 'Coma',
'https://coma.massapro.com',
  true, 2
);
```

## 3.3 Brand Resolver — getEffectiveBrand(context)

The resolver is the single point of brand resolution for the entire codebase. It accepts a context object (request headers, optional chapterId, optional countryId) and returns a BrandContext containing the resolved brand and (if parentLogoUrl is set) the parent brand. The fallback chain is: explicit chapter brand → explicit country brand → host-derived brand → env default (BRAND_DEFAULT_SLUG). All 185 files currently containing "AI Salon" strings will eventually call this resolver — either directly (server components) or via a prop passed from a server component (client components).

```
// src/lib/brand/types.ts
export interface Brand {
  id: string; slug: string; name: string; wordmark: string; tagline: string | null;
  domain: string; siteUrl: string; defaultChapterSlug: string | null;
  fromName: string; fromEmail: string; replyTo: string | null; supportEmail: string | null;
  logoUrl: string | null; logoDarkUrl: string | null; faviconUrl: string | null;
  emailLogoUrl: string | null; loginHeroUrl: string | null; loginBannerUrl: string | null;
  primaryColor: string | null; secondaryColor: string | null;
  accentColor: string | null; gradientCss: string | null;
  parentLogoUrl: string | null; parentLogoPosition: 'ABOVE' | 'BESIDE' | 'FOOTER_ONLY' |
'HIDDEN';
  parentBrandName: string | null; parentBrandUrl: string | null;
}

export interface BrandContext {
  brand: Brand;
  parent: Brand | null;           // resolved parent (if parentLogoUrl set)
  isParent: boolean;              // true if this brand IS the parent (e.g. Coma itself)
  source: 'chapter' | 'country' | 'host' | 'env-default';
}

// src/lib/brand/resolver.ts
import { headers } from 'next/headers';
import { prisma } from '@/lib/prisma';
```

```
import { resolveBrandSlugByHost } from './host-resolver';

export async function getEffectiveBrand(
  ctx: { chapterId?: string; countryId?: string; host?: string } = {}
): Promise<BrandContext> {
  const h = await headers();
  const host = ctx.host || h.get('x-resolved-brand-host') || h.get('host') || '';

  // 1. Explicit chapter brand (rare — only set if chapter was force-reassigned)
  if (ctx.chapterId) {
    const chapter = await prisma.chapter.findUnique({
      where: { id: ctx.chapterId },
      include: { country: { include: { brand: true } } },
    });
    if (chapter?.country?.brand) {
      return finalize(chapter.country.brand, 'chapter');
    }
  }

  // 2. Country brand
  if (ctx.countryId) {
    const country = await prisma.country.findUnique({
      where: { id: ctx.countryId }, include: { brand: true },
    });
    if (country?.brand) return finalize(country.brand, 'country');
  }

  // 3. Host-derived brand
  const slug = resolveBrandSlugByHost(host);
  if (slug) {
    const brand = await prisma.brand.findUnique({ where: { slug } });
    if (brand) return finalize(brand, 'host');
  }

  // 4. Env default
  const defaultSlug = process.env.BRAND_DEFAULT_SLUG || 'aisalon';
  const brand = await prisma.brand.findUnique({ where: { slug: defaultSlug } });
  if (!brand) throw new Error(`Default brand "${defaultSlug}" not found in DB`);
  return finalize(brand, 'env-default');
}

async function finalize(brand: Brand, source: string): Promise<BrandContext> {
  let parent: Brand | null = null;
  if (brand.parentLogoUrl && brand.parentBrandName) {
    // Resolve parent by slug derived from parentBrandName (or store parentBrandId
directly)
    parent = await prisma.brand.findFirst({
      where: { name: brand.parentBrandName },
    });
  }
  return {
    brand, parent,
    isParent: !brand.parentLogoUrl,
    source: source as BrandContext['source'],
  };
}
```

## 3.4 Host Resolver

The host resolver is a pure function (no Prisma call) used by both the middleware (for edge-friendly fast lookup) and the brand resolver. It reads a comma-separated BRAND_HOST_MAP env var (e.g., "coma.massapro.com:coma,aisalon.massapro.com:aisalon,www.aisalon.massapro.com:aisalon") and falls back to a subdomain-based heuristic for chapter subdomains (nyc.aisalon.massapro.com → aisalon). The map is the source of truth for production hosts; the heuristic is a fallback for future chapter subdomains without env-var updates.

```
// src/lib/brand/host-resolver.ts
export function resolveBrandSlugByHost(host: string): string | null {
  if (!host) return null;
  const hostname = host.split(':')[0].toLowerCase();

  // 1. Explicit map (production source of truth)
  const map = parseHostMap(process.env.BRAND_HOST_MAP || '');
  if (map[hostname]) return map[hostname];

  // 2. Subdomain heuristic: *.aisalon.massapro.com → aisalon
  const parentDomain = hostname.split('.').slice(-3).join('.');
  if (map[parentDomain]) return map[parentDomain];

  // 3. localhost dev override
  if (hostname === 'localhost') {
    return process.env.BRAND_DEFAULT_SLUG || 'aisalon';
  }

  return null;
}

function parseHostMap(s: string): Record<string, string> {
  const out: Record<string, string> = {};
  for (const pair of s.split(',').map(x => x.trim()).filter(Boolean)) {
    const [h, slug] = pair.split(':').map(x => x.trim());
    if (h && slug) out[h.toLowerCase()] = slug;
  }
  return out;
}
```

## 3.5 Deprecation of Existing Brand-Image Abstraction

The existing SiteSetting keys K_FAVICON, K_LOGIN_HERO, K_LOGIN_BANNER, K_EMAIL_LOGO and the corresponding ChapterSetting keys are **not removed** — they remain as a per-chapter override layer on top of the new Brand fields. The new 5-tier fallback chain (in getEffectiveBrandImages(brand, chapterId), which replaces the existing getEffectiveBrandImages(chapterId)) is: (1) ChapterSetting[chapterId, key] → (2) Brand[key] → (3) SiteSetting[key] → (4) EMAIL_BRAND_LOGO_URL env var (email-logo only) → (5) hardcoded DEFAULTS constants. The Brand tier is new; the other four tiers preserve

backward compatibility for any chapter that already has a custom image override stored in ChapterSetting.

# 4. Host-Based Routing & Multi-Tenant Dispatch

## 4.1 Middleware Changes

The existing middleware (src/middleware.ts) handles UTM referral tracking only. We extend it with TWO new responsibilities: (a) the coma.massapro.com path-split (/login* → Next.js app, everything else → parallel Coma agent's upstream — see §2.5.4 for the contract) and (b) the brand resolution via the 4-tier chain (URL param → host → session → env default — see §2.5.2). Both are gated by env vars so they are no-ops until enabled. The existing UTM-tracking logic is untouched and continues to run for every request that reaches the Next.js app.

```
// src/middleware.ts — diff (additions only, see §2.5.4 + §2.5.2 for the contract)
import { NextResponse, NextRequest } from 'next/server';
import { resolveBrandSlugByHost } from '@/lib/brand/host-resolver';

const COMA_HOSTS = new Set(['coma.massapro.com', 'www.coma.massapro.com']);
const PLATFORM_PATHS = /^\/(login|api|_next|brand-assets)(\/|$)/;

export async function middleware(req: NextRequest) {
  const host = (req.headers.get('x-forwarded-host') ||
                req.headers.get('host') || '').toLowerCase();

  // === (1) Path-split for coma.massapro.com (see §2.5.4) ===
  // Routes non-/login paths on the coma host to the parallel Coma agent's upstream.
  // Gated by COMA_PARALLEL_UPSTREAM env var — if unset, no split happens.
  if (COMA_HOSTS.has(host) && !PLATFORM_PATHS.test(req.nextUrl.pathname)) {
    const upstream = process.env.COMA_PARALLEL_UPSTREAM;
    if (upstream) {
      const url = new URL(req.nextUrl.pathname + req.nextUrl.search, upstream);
      return NextResponse.rewrite(url);
    }
    // If upstream not set, fall through to Next.js (which will 404 for unknown paths)
  }

  // === (2) Brand resolution (see §2.5.2 — 4-tier chain) ===
  // Gated by BRAND_DOMAIN_RESOLVE_ENABLED feature flag for instant rollback.
  if (process.env.BRAND_DOMAIN_RESOLVE_ENABLED === 'true') {
    // Tier 1: URL param ?brand=<slug> (highest priority)
    const urlBrand = req.nextUrl.searchParams.get('brand');
    // Tier 2: Host-based resolution
    const hostBrand = resolveBrandSlugByHost(host);
    const resolvedSlug = urlBrand || hostBrand ||
                         process.env.BRAND_DEFAULT_SLUG || 'aisalon';
    if (resolvedSlug) {
      const res = NextResponse.next();
      res.headers.set('x-resolved-brand-slug', resolvedSlug);
      req.headers.set('x-resolved-brand-slug', resolvedSlug);
      // Note: Tier 3 (user session) and Tier 4 (env default) are handled
```

```
      // downstream in getBrandForRequest() — the middleware only does the
      // cheap URL+host lookup. Session lookup requires DB access which is
      // too slow for edge middleware.
      return res;
    }
  }

  // === (3) EXISTING: UTM tracking (unchanged) ===
  // ... existing UTM cookie + ReferralVisit logic stays here ...

  return NextResponse.next();
}

export const config = {
  matcher: ["/((?!_next/static|_next/image|favicon\.ico|robots\.txt|sitemap\.xml).*)"],
};
export const runtime = "nodejs"; // unchanged — already Node runtime for Prisma
```

## 4.2 Domain → Brand Resolution Table

The table below is the canonical production routing matrix. The BRAND_HOST_MAP env var must mirror this table exactly. Unknown hosts fall through to the env default (AI Salon).

**Table 1: Host → Brand resolution matrix**

| Host | Brand Slug | Parent Logo | Notes |
|---|---|---|---|
| coma.massapro.com | coma | none (is parent) | Production parent-brand host. New users get brandId='coma'. |
| www.coma.massapro.com | coma | none | WWW redirect target — also mapped. |
| aisalon.massapro.com | aisalon | Coma (above) | Existing chapters host. Zero behavioral change for existing users. |
| www.aisalon.massapro.com | aisalon | Coma (above) | WWW variant. |
| nyc.aisalon.massapro.com | aisalon | Coma (above) | Future chapter subdomain (resolved via heuristic). |
| <anything>.aisalon.massapro.com | aisalon | Coma (above) | Subdomain heuristic — any 3-part hostname ending in aisalon.massapro.com. |

| Host | Brand Slug | Parent Logo | Notes |
|---|---|---|---|
| localhost | aisalon | Coma (above) | Dev default. Override with BRAND_DEFAULT_SLUG=coma to test Coma locally. |
| localhost:3000 | aisalon | Coma (above) | Same — port stripped before lookup. |
| (unknown host) | (env default) | (per default brand) | Fallback — logs warning, falls through to BRAND_DEFAULT_SLUG. |

### 4.3 Server-Side Consumption

Server components and route handlers read the resolved brand via `getEffectiveBrand()`. The resolver reads the `x-resolved-brand-slug` request header set by the middleware, then queries the `Brand` table by slug. Because the middleware has already done the host→slug mapping, the resolver's Prisma query is a single primary-key lookup — fast enough to call per-request without caching. For high-traffic routes (e.g., the login page), the resolver result can be cached per-request via React's `cache()` wrapper or a per-request Map.

Route handlers (`src/app/api/**/route.ts`) read the brand via the same helper. API routes that create users (e.g., `/api/auth/signup`) read the brand once and stamp `brandId` on the new `User` row, so the user is permanently associated with the brand they registered under. Existing users (who registered before this refactor) have `brandId = 'brand_aisalon'` from the backfill — they continue to see AI Salon regardless of which host they visit, which matches the requirement that existing users see no change.

### 4.4 Client-Side Consumption

Client components receive the brand as a prop from their parent server component. The root layout (`src/app/layout.tsx`) calls `getEffectiveBrand()` once per request and passes the `BrandContext` to a `BrandProvider` React Context at the root of the tree. Client components consume via `useBrand()`. This avoids prop-drilling through deeply nested trees and keeps the brand object referentially stable across client-side navigations within a single brand.

For client-side navigations across brands (rare — would require the user to navigate from `aisalon.massapro.com` to `coma.massapro.com` in the same SPA session), the brand context is re-fetched on the new host. In practice this is a full page reload because the hosts are different origins, so the context is always fresh from the server.

### 4.5 Edge Cases

- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]

### 4.6 Env Var Additions

Three new env vars are added to .env.example. The BRAND_HOST_MAP is the production source of truth for host→slug mapping and must be kept in sync with Table 1.

```
# .env.example — additions

# Brand resolution
BRAND_DEFAULT_SLUG=aisalon
# Feature flag — set to "true" to enable host-based brand dispatch.
# When "false" (or unset), all requests use BRAND_DEFAULT_SLUG.
BRAND_DOMAIN_RESOLVE_ENABLED=true

# Host → brand slug map (comma-separated, host:slug pairs).
# Mirrors Table 1 of the Brand-Field Platform Plan.
BRAND_HOST_MAP=coma.massapro.com:coma,www.coma.massapro.com:coma,aisalon.m
assapro.com:aisalon,www.aisalon.massapro.com:aisalon
```

In next.config.ts, the images.remotePatterns array must include coma.massapro.com (alongside the existing aisalon.massapro.com and *.massapro.com entries) so the Next.js Image component can load Coma-hosted brand assets. This is already partially in place (*.massapro.com covers it), but the explicit entry is added for clarity and forward-compat with future non-massapro.com client domains.

## 5. Login Page Replication Guide (Brand-Swap)

### 5.1 Current State Recap

The login page is implemented across two files: src/app/login/page.tsx (server component) and src/app/login/login-form.tsx (client component). The server component hardcodes DEFAULT_CHAPTER_SLUG = "tel-aviv" on line 46, imports the AiSalonLogoServer component (from @/components/brand/aisalon-logo-server), and embeds the AI Salon color palette directly in the JSX via inline gradient strings (conic-gradient(from 180deg at 50% 50%,

#FF005A, #820A7D, #004F98, #00E6FF, #FF005A)). The page title template is "Login — AI Salon ${chapterName}", the description meta tag reads "Log in to AI Salon ${chapterName} — the community for AI builders, founders, CMOs and investors in ${chapterName}.", and the footer credits "Platform by MassaPro · Powered by AI Salon". The hero image and banner mark URLs are loaded via getEffectiveBrandImagesBySlug(chapterSlug) from src/lib/chapter-brand-images.ts, which already implements a 3-tier image fallback chain (chapter override → global → defaults) — this is the only existing brand-aware logic on the page.

## 5.2 Target State

The login page becomes **brand-agnostic by design**. A single Next.js route (/login) reads the resolved brand from the request (via the 4-tier chain in §2.5.2) AND the optional chapterSlug search param, then swaps the logo, colors, strings, mascot, and chapter-name display. There is no forked route, no separate codebase, no separate deployment. Per the URL contract in §2.5.1:

- [object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object],[object Object]

Both routes share 100% of the source code — only the brand configuration differs, and that configuration comes from the database, not from environment-specific branches. The ?brand= URL param is the escape hatch for any external client brand (Google, future customers) without DNS provisioning — the brand row just needs to exist in the Brand table.

## 5.3 Step-by-Step Checklist

The login agent (parallel thread) implements the following steps. Each step is independently testable. The order matters — steps 1 and 2 are the contract with the platform agent and must be agreed before the rest begins.

1. Read the resolved brand at the top of page.tsx via getBrandForRequest(). This returns a BrandContext object containing the brand and (if set) the parent brand.
2. Replace AiSalonLogoServer with the new BrandLogoServer component, passing the brand prop. BrandLogoServer is owned by the platform agent and lives at src/components/brand/brand-logo-server.tsx.
3. Replace DEFAULT_CHAPTER_SLUG = "tel-aviv" with brand.defaultChapterSlug. For AI Salon this resolves to "tel-aviv" (no behavioral change). For Coma this resolves to null — the login page then shows a brand-default landing without a chapter context (Coma is not chapter-based).

4. Replace the hardcoded AIS color palette (#FF005A, #00E6FF, #004F98, #820A7D) with brand.primaryColor, brand.secondaryColor, brand.accentColor, and brand.gradientCss injected via inline style attributes.
5. Replace the metadata title template "Login — AI Salon ${chapterName}" with "Login — ${brand.name}${chapterName ? ' ' + chapterName : ''}". Same for the description meta tag and the eyebrow text.
6. Replace the hero copy template strings ("The community for AI builders in ${chapterName}.", "Empowering AI Connections in ${chapterName}") with brand-aware variants. Coma's hero copy will differ from AI Salon's — the login agent writes the copy, sourced from a brand.heroHeadline and brand.heroSubtitle field. If those fields are null, fall back to a generic "Sign in to ${brand.name}".
7. Replace the footer credit "Platform by MassaPro · Powered by AI Salon" with a brand-aware credit. For AI Salon: "Platform by Coma · Powered by AI Salon". For Coma: "Platform by Coma". The pattern is: if brand.parent exists, render "Platform by ${parent.name} · Powered by ${brand.name}"; else render "Platform by ${brand.name}".
8. Render the Coma parent logo above the brand logo when brand.parent is set. The BrandLogoServer component handles this internally — the login agent just passes the brand prop. The parentLogoPosition field (ABOVE / BESIDE / FOOTER_ONLY / HIDDEN) controls placement; ABOVE is the default for AI Salon.
9. Add a favicon override via metadata.icons using brand.faviconUrl. Next.js 16 metadata.icons accepts a URL string or an array of icon descriptors.
10. Add an OG image override via metadata.openGraph.images using brand.loginHeroUrl. This ensures link previews on social media show the correct brand.
11. Test on localhost:3000 with BRAND_DEFAULT_SLUG=coma env var. Verify the Coma logo, Coma colors, Coma page title, and Coma hero copy all render. Then test with BRAND_DEFAULT_SLUG=aisalon (or unset) and verify the AI Salon brand renders as today.
12. Deploy to a staging URL and verify both coma.massapro.com/login and aisalon.massapro.com/login render the correct brand. Use curl or a browser to inspect the HTML — the page title and the logo URL must match the host.

## 5.4 Code Snippet — page.tsx Top Section

The login agent's first edit to page.tsx replaces the hardcoded defaults with a brand lookup. The full file is longer; this snippet shows only the top section that changes. The rest of the file (form, validation, submit handler) is unchanged because it already operates on the chapterSlug, which is now sourced from the brand.

```
// src/app/login/page.tsx — top section after refactor
import { getBrandForRequest } from "@/lib/brand/resolver";
import { BrandLogoServer } from "@/components/brand/brand-logo-server";
import { getEffectiveBrandImagesBySlug } from "@/lib/chapter-brand-images";
```

```
// NOTE: getBrandForRequest() already implements the 4-tier resolution chain
// from §2.5.2 — URL ?brand= param > host > session > env default.
// We do NOT re-read ?brand= here; the resolver handles it.

export async function generateMetadata({ searchParams }) {
  const { brand } = await getBrandForRequest();
  // chapterSlug comes from URL (e.g. ?chapterSlug=mtl) or falls back to the
  // brand's defaultChapterSlug field (e.g. "tel-aviv" for aisalon, null for coma)
  const chapterSlug = searchParams.chapterSlug || brand.defaultChapterSlug;
  const chapterName = chapterSlug
    ? await getChapterName(chapterSlug)   // existing helper
    : "";
  const title = chapterName
    ? `Login — ${brand.name} ${chapterName}`
    : `Login — ${brand.name}`;
  const description = `Log in to ${brand.name}${chapterName ? " " + chapterName : ""}
— ${brand.tagline || "the community platform"}.`;
  return {
    title,
    description,
    icons: { icon: brand.faviconUrl || "/favicon.ico" },
    openGraph: { images: [brand.loginHeroUrl].filter(Boolean) },
  };
}

export default async function LoginPage({ searchParams }) {
  const { brand, parent } = await getBrandForRequest();
  const chapterSlug = searchParams.chapterSlug || brand.defaultChapterSlug;
  const settings = await getEffectiveBrandImagesBySlug(chapterSlug);
  const chapterName = chapterSlug ? await getChapterName(chapterSlug) : "";

  // Examples of resolved brand for different URLs (all on the SAME code path):
  //   /login                                  → brand=aisalon (host default)
  //   /login?chapterSlug=mtl                  → brand=aisalon, chapter=Montreal
  //   /login?brand=coma                       → brand=coma (URL override)
  //   /login?brand=google&chapterSlug=tlv     → brand=google, chapter=Tel Aviv
  //                                           (renders Google logo + Coma parent above)

  return (
    <div style={{ background: brand.gradientCss || undefined }}>
      {/* ... rest of the page ... */}
      <BrandLogoServer
        brand={brand}
        parent={parent}
        variant="horizontal-tagline"
        color="white"
        markSrc={settings.loginBanner}
      />
      {/* ... hero copy uses brand.heroHeadline / brand.heroSubtitle ...
          If chapterName is non-empty, render it as the chapter eyebrow above the headline.
*/}
    </div>
  );
}
```

## 5.5 Code Snippet — BrandLogoServer Component

This component is owned by the platform agent (it is part of the boundary contract). It renders the brand's primary logo and, when a parent brand exists, renders the parent logo above (or beside, or in the footer-only — controlled by brand.parentLogoPosition). The login agent consumes this component without modification.

```
// src/components/brand/brand-logo-server.tsx
import Image from "next/image";
import type { Brand, BrandContext } from "@/lib/brand/types";

interface Props {
  brand: Brand;
  parent: Brand | null;
  variant?: "horizontal-tagline" | "mark-only" | "stacked";
  color?: "white" | "dark";
  className?: string;
  markSrc?: string;   // override the primary logo URL (for chapter-specific marks)
}

export function BrandLogoServer({
  brand, parent, variant = "horizontal-tagline",
  color = "dark", className, markSrc,
}: Props) {
  const logoSrc = markSrc || brand.logoUrl;
  if (!logoSrc) return null;

  // No parent or parent hidden → just render the brand logo
  if (!parent || brand.parentLogoPosition === "HIDDEN") {
    return <BrandLogo src={logoSrc} alt={brand.name} variant={variant}
className={className} />;
  }

  // Footer-only parent → brand logo in header, parent logo in footer
  // (footer rendering is the consumer's responsibility; we just return the brand logo here)
  if (brand.parentLogoPosition === "FOOTER_ONLY") {
    return <BrandLogo src={logoSrc} alt={brand.name} variant={variant}
className={className} />;
  }

  // ABOVE (default) or BESIDE → render parent + brand logos together
  const isAbove = brand.parentLogoPosition === "ABOVE";
  return (
    <div className={isAbove ? "flex flex-col items-center gap-2" : "flex items-center
gap-3"}>
      {parent.logoUrl && (
        <Image
          src={parent.logoUrl}
          alt={parent.name}
          width={80}
          height={24}
          style={{ width: 80, height: "auto", opacity: 0.85 }}
          priority
        />
      )}
      <BrandLogo src={logoSrc} alt={brand.name} variant={variant}
```

```
className={className} />
    </div>
  );
}

function BrandLogo({ src, alt, variant, className }: {
  src: string; alt: string; variant: string; className?: string;
}) {
  const width = variant === "mark-only" ? 48 : 150;
  return (
    <Image
      src={src} alt={alt}
      width={width} height={48}
      style={{ width: width, height: "auto" }}
      className={className}
      priority
    />
  );
}
```

## 5.6 DNS / CNAME Checklist

Before coma.massapro.com/login can serve the Coma brand, the DNS and Vercel configuration must be in place. This is a deployment task, not a code task. The platform agent owns this checklist; the login agent does not need to perform any DNS work.

1. Add coma.massapro.com as a domain in the Vercel project settings (Project → Settings → Domains → Add). Vercel generates a DNS verification record.
2. In the MassaPro DNS provider (wherever massapro.com is managed), add either a CNAME record coma.massapro.com → cname.vercel-dns.com OR an A record coma.massapro.com → 76.76.21.21 (Vercel's anycast IP). CNAME is preferred.
3. Wait for DNS propagation (typically 5-30 minutes for new records; verify with dig coma.massapro.com or nslookup).
4. Verify in Vercel that the domain shows a green "Valid Configuration" status. If it shows "Invalid Configuration", double-check the DNS record type and value.
5. Set the BRAND_HOST_MAP env var in Vercel to include coma.massapro.com:coma (per Section 4.6). Deploy the new code.
6. Visit https://coma.massapro.com/login in a browser. Verify the Coma logo, Coma colors, "Coma" in the page title, and that no AI Salon strings appear in the rendered HTML.
7. Optionally configure a redirect from www.coma.massapro.com → coma.massapro.com in Vercel (automatic when adding the apex domain).

## 5.7 Acceptance Criteria

The login page replication is complete when all of the following are true. The login agent signs off on items 1-5; the platform agent signs off on items 6-9.

- [object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object]

# 6. Email & Signature Rebrand (with Coma Coexistence)

## 6.1 Two Email Code Paths

Per the codebase map, the email subsystem has two parallel code paths that both need rebranding. The first is src/lib/email.ts (528 lines, 28 "AI Salon" occurrences), which contains the nodemailer SMTP transport and four transactional email builders (sendPasswordEmail, sendRsvpConfirmationEmail, sendChapterOnboardingEmail, sendChapterProvisionedEmail). Each builder has hardcoded "AI Salon" strings in the subject, body, plain-text signature, and HTML signature, plus a hardcoded from-address fallback of "AI Salon Chat <chat@aisalon.massapro.com>" on line 68. The second path is the orchestrator/campaign flow (src/lib/email-orchestrator/templates.ts with 18 occurrences + src/lib/email-orchestrator/seed.ts with 5 + src/lib/email/render-unified.ts with 2), which uses a SHELL HTML wrapper with the lowercase wordmark "aisalon", a footer "AI Salon {{chapter_name}} · Empowering AI Connections", and the URL https://aisalon.massapro.com. Both paths converge on the same renderUnifiedEmail() renderer for token replacement and logo injection.

## 6.2 New Email Tokens

The renderer's token replacer (in src/lib/email/render-unified.ts) currently handles {{chapter_name}} and a handful of camelCase tokens (e.g., {{speakerName}}, {{eventDate}}). We add nine new brand tokens. All brand tokens use the {{brand_*}} and {{parent_brand_*}} namespaces to avoid collisions with existing template-specific tokens. The replacer is case-sensitive and matches both snake_case ({{brand_name}}) and camelCase ({{brandName}}) variants for forward compatibility.

**Table 2: New email tokens**

| Token | Source | Example (AI Salon) | Example (Coma) |
|---|---|---|---|
| {{brand_name}} | brand.name | AI Salon | Coma |
| {{brand_tagline}} | brand.tagline | Empowering AI Connections | The platform behind AI Salon |
| {{brand_url}} | brand.siteUrl | https://aisalon.massapro.com | https://coma.massapro.com |
| {{brand_wordmark}} | brand.wordmark | aisalon | coma |
| {{brand_logo_url}} | brand.emailLogoUrl | .../aisalon-logo.webp | .../coma_trans.png |
| {{brand_primary_color}} | brand.primaryColor | #FF005A | #0F172A |
| {{brand_support_email}} | brand.supportEmail | support@aisalon.massapro.com | support@coma.massapro.com |
| {{parent_brand_name}} | parent.name | Coma | (empty — no parent) |
| {{parent_brand_logo_url}} | parent.logoUrl | https://coma.massapro.com/assets/coma_trans.png | (empty) |
| {{parent_brand_url}} | parent.siteUrl | https://coma.massapro.com | (empty) |

## 6.3 Updated SHELL in templates.ts

The SHELL function in src/lib/email-orchestrator/templates.ts wraps every orchestrator email. The current implementation hardcodes the wordmark, footer line, and URL. The refactored version replaces all hardcoded values with tokens. The tokens are then resolved by renderUnifiedEmail() at send time using the resolved brand context. This means the same template HTML serves all brands — the brand context is injected at render time, not stored in the template.

```
// src/lib/email-orchestrator/templates.ts — refactored SHELL
const SHELL = (inner: string): string => `<!DOCTYPE html>
<html lang="en">
<head>
  <title>{{brand_name}} {{chapter_name}}</title>
</head>
<body ...>
  <div ...>
    <div data-brand-header ...>{{brand_wordmark}}</div>
    <br><br>
    ${inner}
    <hr data-brand-content-end .../>
    <p ...>
      {{brand_name}} {{chapter_name}} · {{brand_tagline}}
      <a href="{{brand_url}}" ...>{{brand_url}}</a>
```

```
    </p>
  </div>
</body></html>`;

// Each of the 5 stage templates now ends with:
// — The {{brand_name}} {{chapter_name}} team
// (was: — The AI Salon {{chapter_name}} team)
```

## 6.4 Coma Coexistence in Email Header

The buildLogoBlock() function in templates.ts currently renders a single brand logo as a 150px-wide image. The refactored version renders a two-row table when brand.parentLogoUrl is set and brand.parentLogoPosition is ABOVE or BESIDE. The parent logo is rendered at 80px wide (smaller, denoting the parent brand); the chapter brand logo is rendered at 150px wide (the primary mark). When parentLogoPosition is FOOTER_ONLY, the parent logo is not rendered in the header — it appears only in the signature. When HIDDEN, no parent logo appears anywhere (used by the Coma brand itself, which has no parent).

```
// src/lib/email-orchestrator/templates.ts — refactored buildLogoBlock
function buildLogoBlock(brand: Brand, parent: Brand | null): string {
  const chapterLogo = `<img src="${brand.emailLogoUrl}" alt="${brand.name}"
    width="150" style="width:150px;height:auto;..."/>`;

  // No parent or HIDDEN → single logo (current behavior)
  if (!parent || !parent.logoUrl || brand.parentLogoPosition === "HIDDEN") {
    return chapterLogo;
  }

  // FOOTER_ONLY → header shows only chapter logo; parent goes in signature
  if (brand.parentLogoPosition === "FOOTER_ONLY") {
    return chapterLogo;
  }

  // ABOVE → two-row table, parent on top (smaller)
  if (brand.parentLogoPosition === "ABOVE") {
    return `<table role="presentation" cellpadding="0" cellspacing="0"
      style="margin:0 auto;border-collapse:collapse;">
      <tr><td style="text-align:center;padding-bottom:8px;">
        <img src="${parent.logoUrl}" alt="${parent.name}"
          width="80" style="width:80px;height:auto;opacity:0.85;"/>
      </td></tr>
      <tr><td style="text-align:center;">
        ${chapterLogo}
      </td></tr>
    </table>`;
  }

  // BESIDE → two-column table, parent on left (smaller)
  return `<table role="presentation" cellpadding="0" cellspacing="0"
    style="margin:0 auto;border-collapse:collapse;">
    <tr>
      <td style="padding-right:16px;vertical-align:middle;">
```

```
      <img src="${parent.logoUrl}" alt="${parent.name}"
        width="80" style="width:80px;height:auto;opacity:0.85;"/>
      </td>
      <td style="vertical-align:middle;">
        ${chapterLogo}
      </td>
    </tr>
  </table>`;
}
```

## 6.5 Email Signature Rebrand

Both the transactional signatures (in email.ts) and the orchestrator SHELL footer (in templates.ts) are rebranded. The pattern is: when the brand has a parent, the signature credits the parent — "Coma · https://coma.massapro.com" appears below the chapter team line. When the brand has no parent (Coma itself), the signature credits the brand directly — "Coma · https://coma.massapro.com". The from-address is also rebranded: transactional emails use brand.fromName and brand.fromEmail resolved via getEffectiveBrand(), with the existing SMTP_FROM env var as an override tier (for testing or bulk-sender configurations that require a fixed from-address).

```
// src/lib/email.ts — refactored signature + from-address

// NEW: from-address resolution (replaces hardcoded fallback on line 68)
async function resolveFromAddress(brand: Brand): Promise<string> {
  // Tier 1: explicit per-message override (campaigns, test sends)
  // Tier 2: SMTP_FROM env var (operator-configured)
  if (process.env.SMTP_FROM) return process.env.SMTP_FROM;
  // Tier 3: brand.fromName + brand.fromEmail
  return `${brand.fromName} <${brand.fromEmail}>`;
}

// Refactored sendPasswordEmail signature
async function sendPasswordEmail(opts: { to: string; password: string;
  chapterName: string; brand: Brand; parent: Brand | null; }) {
  const { brand, parent, chapterName, ...rest } = opts;
  const from = await resolveFromAddress(brand);

  // Plain-text signature (was: hardcoded "AI Salon {chapterName} team")
  const parentCredit = parent
    ? `${parent.name} · ${parent.siteUrl}`
    : `${brand.name} · ${brand.siteUrl}`;
  const plainSignature = `— The ${brand.name} ${chapterName} team\n${parentCredit}
`;

  // HTML signature
  const htmlSignature = `${brand.name} ${chapterName} · ${brand.tagline || ""}<br/>
    <a href="${parent ? parent.siteUrl : brand.siteUrl}" style="color:#999;">
      ${parent ? parent.name : brand.name}
    </a>`;

  // ... rest of sendMail() call unchanged ...
```

```
}
```

## 6.6 Unsubscribe Footer

The unsubscribe footer in render-unified.ts currently reads "You received this email because you are a member of AI Salon {{chapter_name}}." The refactored version replaces the hardcoded "AI Salon" with the {{brand_name}} token. The unsubscribe link itself (/api/email/unsubscribe?token=...) is host-relative and resolves correctly on whichever host the user clicks from — but to be safe, we prefix it with {{brand_url}} so the unsubscribe link always points to the brand's canonical host, even if the email is forwarded or viewed in a mail client that doesn't preserve the original host context.

```
// src/lib/email/render-unified.ts — refactored unsubscribe footer
const UNSUBSCRIBE_FOOTER = `<p style="...font-size:11px;color:#999;">
  You received this email because you are a member of
  {{brand_name}} {{chapter_name}}.
  <a href="{{brand_url}}/api/email/unsubscribe?token={{unsubscribe_token}}"
     style="color:#999;">Unsubscribe</a>.
</p>`;
```

## 6.7 Seed Migration

The EmailTemplate2 table currently stores 5 stage templates (Awareness, RSVP, Reminder, Post-Event, Re-engagement) seeded by src/lib/email-orchestrator/seed.ts. The bodyHtml column of each row contains hardcoded "AI Salon" strings (because the seed was written before this refactor). A one-time migration script (scripts/migrate-email-templates-to-brand-tokens.ts) iterates every EmailTemplate2 row and replaces the hardcoded strings with the new tokens. The script is idempotent — running it twice has no effect because the second run finds no hardcoded strings to replace.

```
// scripts/migrate-email-templates-to-brand-tokens.ts
import { prisma } from "../src/lib/prisma";

const REPLACEMENTS: Array<[RegExp, string]> = [
  [/AI Salon {{chapter_name}}/g, "{{brand_name}} {{chapter_name}}"],
  [/-- The AI Salon {{chapter_name}} team/g, "— The {{brand_name}} {{chapter_name}}
team"],
  [/Empowering AI Connections/g, "{{brand_tagline}}"],
  [/https://aisalon.massapro.com/g, "{{brand_url}}"],
  [/<div data-brand-header[^>]*>aisalon</div>/g, '<div data-brand-
header>{{brand_wordmark}}</div>'],
  [/alt="AI Salon"/g, 'alt="{{brand_name}}"'],
];

async function main() {
  const templates = await prisma.emailTemplate2.findMany();
  let updated = 0;
  for (const tpl of templates) {
```

```
    let bodyHtml = tpl.bodyHtml;
    let signatureHtml = tpl.signatureHtml || "";
    for (const [re, replacement] of REPLACEMENTS) {
      bodyHtml = bodyHtml.replace(re, replacement);
      signatureHtml = signatureHtml.replace(re, replacement);
    }
    if (bodyHtml !== tpl.bodyHtml || signatureHtml !== (tpl.signatureHtml || "")) {
      await prisma.emailTemplate2.update({
        where: { id: tpl.id },
        data: { bodyHtml, signatureHtml },
      });
      updated++;
      console.log(`  Migrated ${tpl.name} (${tpl.id})`);
    }
  }
  console.log(`Done. ${updated}/${templates.length} templates migrated.`);
}
main().then(() => process.exit(0));
```

### 6.8 Acceptance Criteria

The email rebrand is complete when all of the following are true:

- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]

## 7. Admin UI — Brand Management Screen

### 7.1 Route + Tab Integration

A new top-level admin route /admin/brand is added. The route is registered in the ALL_TABS array in src/components/ais/admin-tabs-def.ts with a SUPER_ADMIN-only role filter — CHAPTER_ORGANIZER, CO_HOST, SPEAKER, and MEMBER roles do not see the tab. The tab uses the Palette icon from lucide-react. The existing GlobalBrandImagesEditor (currently living on the /admin/chapters page) is moved into the new Brand page as a sub-section; a redirect is left at the old location so existing bookmarks still work.

```
// src/components/ais/admin-tabs-def.ts — addition to ALL_TABS
import { Palette } from "lucide-react";

export const ALL_TABS = [
```

```
  // ... existing tabs ...
  {
    href: "/admin/brand",
    label: "Brand",
    icon: Palette,
    match: "/admin/brand",
    roles: ["SUPER_ADMIN"],   // ← only SUPER_ADMIN sees this tab
  },
  // ... rest of existing tabs ...
];
```

## 7.2 Two-Tab Page Structure

The Brand Management page has two internal tabs: **Brands** (list + edit per brand) and **Chapter Overrides** (per-chapter brand field overrides). The Brands tab is the primary surface — SUPER_ADMIN uses it to add new brands, edit existing brand fields, and toggle brand active/inactive. The Chapter Overrides tab is the per-chapter customization surface — a CHAPTER_ORGANIZER (if granted access) or SUPER_ADMIN can override any brand field for a specific chapter, stored as `ChapterSetting` rows with the `brand.override.` key prefix.

## 7.3 Brands Tab Wireframe

The Brands tab displays a table of all `Brand` rows, sorted by `sortOrder` then `name`. Columns: Slug, Name, Domain, Parent Brand, Active, Actions (Edit / Deactivate). A "New Brand" button above the table opens a modal or drawer with the full brand form. Clicking Edit on a row opens the same form pre-filled with the brand's current values.

The brand form contains the following fields, grouped into four sections (Identity, Domain & Email, Images, Colors & Parent). All fields are optional unless marked required. Image fields accept either a URL (pasted) or a file upload (stored in `/public/uploads/brand-assets/` with a timestamped filename, matching the existing pattern in `/api/admin/brand-images`).

**Table 3: Brand form fields**

| Section | Field | Type | Required | Notes |
|---|---|---|---|---|
| Identity | slug | text | Yes | Unique, lowercase, no spaces (e.g. 'coma') |
| Identity | name | text | Yes | Display name (e.g. 'Coma') |
| Identity | wordmark | text | Yes | Lowercase identifier for email header (e.g. 'coma') |

| Section | Field | Type | Required | Notes |
|---|---|---|---|---|
| Identity | tagline | text | No | Email footer line (e.g. 'Empowering AI Connections') |
| Identity | description | textarea | No | Internal notes about this brand |
| Domain & Email | domain | text | Yes | Canonical host (e.g. 'coma.massapro.com') |
| Domain & Email | altDomains | comma-input | No | Additional hosts (comma-separated) |
| Domain & Email | siteUrl | text | Yes | Full URL (e.g. 'https://coma.massapro.com') |
| Domain & Email | defaultChapterSlug | text | No | Chapter slug to land on; null = brand-default landing |
| Domain & Email | fromName | text | Yes | Email from-name (e.g. 'Coma') |
| Domain & Email | fromEmail | text | Yes | Email from-address (e.g. 'no-reply@coma.massapro.com') |
| Domain & Email | replyTo | text | No | Reply-to address (defaults to fromEmail) |
| Domain & Email | supportEmail | text | No | Support contact (e.g. 'support@coma.massapro.com') |
| Images | logoUrl | file+url | No | Primary mark (light-bg variant) |
| Images | logoDarkUrl | file+url | No | Dark-bg variant (optional) |
| Images | faviconUrl | file+url | No | Browser tab icon |
| Images | emailLogoUrl | file+url | No | Email header logo (150px wide) |

| Section | Field | Type | Required | Notes |
|---|---|---|---|---|
| Images | loginHeroUrl | file+url | No | Login page hero image |
| Images | loginBannerUrl | file+url | No | Login page banner mark |
| Colors | primaryColor | color-picker | No | Hex (e.g. #FF005A) |
| Colors | secondaryColor | color-picker | No | Hex |
| Colors | accentColor | color-picker | No | Hex |
| Colors | gradientCss | textarea | No | Raw CSS for gradient (e.g. 'conic-gradient(...)') |
| Parent | parentLogoUrl | url | No | Parent brand logo URL (null on Coma) |
| Parent | parentLogoPosition | dropdown | No | ABOVE (default) / BESIDE / FOOTER_ONLY / HIDDEN |
| Parent | parentBrandName | text | No | Parent brand name (denormalized for email rendering) |
| Parent | parentBrandUrl | url | No | Parent brand canonical URL |
| Status | isActive | toggle | Yes | Inactive brands 404 on their domain |
| Status | sortOrder | number | No | Display order in admin (default 0) |

## 7.4 Chapter Overrides Tab Wireframe

The Chapter Overrides tab allows per-chapter customization of brand fields. A dropdown at the top selects the chapter (fetched from the Chapter table, scoped to the user's role — CHAPTER_ORGANIZER sees only their own chapter, SUPER_ADMIN sees all). Once a chapter is selected, the form shows every overridable brand field (logoUrl, loginHeroUrl, loginBannerUrl, emailLogoUrl, fromName, fromEmail, replyTo) with the current effective value (inherited from the brand) and an override input. Setting an override stores a ChapterSetting row with key

brand.override.<fieldName> and the override value. Clearing an override deletes the row, reverting to the brand default.

The override mechanism reuses the existing ChapterSetting table — no schema change is needed. The getEffectiveBrandImages() resolver (now extended to also resolve non-image brand fields) checks for brand.override.* keys first, then falls back to the Brand field, then to SiteSetting, then to the hardcoded defaults. This means a chapter can override any combination of fields — for example, NYC could use the AI Salon brand identity but with a custom NYC-specific login hero image and a custom from-name "AI Salon NYC".

## 7.5 Code Structure

The Brand Management page follows the existing admin page pattern: a server component (page.tsx) fetches the initial data and passes it to a client component (brand-admin-client.tsx) which handles the interactive UI (form state, modal open/close, optimistic updates). The form itself is a separate component (brand-form.tsx) reused for both create and edit. API routes follow the existing /api/admin/* pattern, scoped to SUPER_ADMIN via the existing permission check.

- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]

## 7.6 Reuse of Existing Editors

The existing GlobalBrandImagesEditor (currently on the /admin/chapters page) is moved into the Brand Management page as a third sub-section ("Global Image Defaults"). This editor manages the SiteSetting rows for the four image keys (favicon, loginHero, loginBanner, emailLogo) — it remains useful as a global override layer above the per-Brand defaults. A redirect is left at the old location: visiting /admin/chapters with the intent to edit global brand images shows a banner "This section has moved to /admin/brand" with a link. The ChapterBrandImagesEditor (per-chapter image overrides) stays on the chapter edit page at /admin/c/[chapterSlug] — it is not moved, because chapter organizers are accustomed to finding it there and the Chapter Overrides tab in /admin/brand is a SUPER_ADMIN-only power-user surface.

### 7.7 Future Client Brand Onboarding

When a new client brand is signed (e.g., a hypothetical "Acme Corp" with domain acme.events), the onboarding flow is: (1) SUPER_ADMIN visits /admin/brand → clicks "New Brand" → fills in Acme's identity, domain, images, colors, and sets parentLogoUrl to the Coma logo (or leaves null if Acme is not a Coma subsidiary). (2) DNS is configured: acme.events → Vercel. (3) The BRAND_HOST_MAP env var is updated to include acme.events:acme. (4) A new deploy is triggered (env var changes require a redeploy on Vercel). (5) The new brand is live at acme.events/login. **Zero code change is required** — the entire flow is data + configuration. This is the key deliverable of the multi-tenant architecture: the platform is one, brands are data.

## 8. Single-Sprint Implementation Plan

### 8.1 Sprint Timeline

The refactor is scoped as a single 10-working-day sprint. Nine PRs are sequenced below; eight are owned by the platform agent (this thread) and one (PR#5) is owned by the parallel login agent. PRs are merged in order; each PR is independently deployable via the feature flag. The total elapsed time is 10 working days (2 calendar weeks) assuming one engineer on each agent thread.

**Table 4: Sprint timeline — 9 PRs over 10 working days**

| Day | PR # | Owner | Title | Description |
|---|---|---|---|---|
| 1-2 | PR#1 | Platform | Schema + migration | Add Brand model + FKs on User/Country + migration SQL + backfill (Section 3 + Section 10). Verify in staging. |
| 2-3 | PR#2 | Platform | Brand resolver + middleware | Add src/lib/brand/{types,resolver,host-resolver}.ts; extend src/middleware.ts with host dispatch; add env vars. Feature flag default off. |

| Day | PR # | Owner | Title | Description |
|---|---|---|---|---|
| 3-5 | PR#3 | Platform | Email rebrand | Update src/lib/email.ts (signatures + from-address), src/lib/email-orchestrator/templates.ts (SHELL + buildLogoBlock), src/lib/email/renderer-unified.ts (tokens + unsubscribe). Run seed migration script. |
| 5-6 | PR#4 | Platform | Admin Brand UI | Add /admin/brand page + form + chapter overrides + API routes. Integrate tab in admin-tabs-def.ts. Move GlobalBrandImagesEditor. |
| 6-7 | PR#5 | Login agent | Login page brand-swap | Implement the 12-step checklist from Section 5.3. This is the parallel agent's only PR. |
| 7-8 | PR#6 | Platform | Metadata refactor | Replace ~150 admin/page.tsx metadata = { title: 'X — AI Salon' } with generateMetadata() reading brand. Automated codemod. |

| Day | PR # | Owner | Title | Description |
|---|---|---|---|---|
| 8-9 | PR#7 | Platform | Logo component swap | Replace AiSalonLogoServer with BrandLogoServer across all consumers. Deprecate (don't delete) the old components. |
| 9 | PR#8 | Platform | Mockup sample-data | Update src/app/admin/mockups/*/sample-data.ts to use brand-derived values instead of hardcoded 'AI Salon Tel Aviv'. |
| 10 | PR#9 | Platform | DNS + prod deploy | Configure coma.massapro.com DNS + Vercel domain. Set BRAND_HOST_MAP env. Enable feature flag. Smoke test on both hosts. |

## 8.2 Dependencies Graph

PR#1 (schema) blocks all others — every downstream PR depends on the Brand model existing. PR#2 (resolver + middleware) blocks PR#3 (email needs the resolver to pass brand context), PR#5 (login needs `getBrandForRequest()` and `BrandLogoServer`), PR#6 (metadata needs the resolver), and PR#7 (logo swap needs the new component, which depends on the Brand type from PR#2). PR#4 (admin UI) is independent of PR#3 — it can be developed in parallel once PR#1 lands. PR#8 (mockups) is last because it depends on PR#7's BrandLogoServer being in place. PR#9 (DNS + prod deploy) is the final cut-over and depends on all prior PRs being merged to main.

Critical path: PR#1 → PR#2 → PR#5 → PR#9 (the login brand-swap is the user-visible deliverable; everything else is infrastructure). PR#3, PR#4, PR#6, PR#7, PR#8 can be parallelized with PR#5 once PR#2 lands, but in practice with one engineer per agent thread they sequence as shown in Table 4. If a second platform engineer is available, PR#3 and PR#4 can be developed in parallel with PR#5, compressing the sprint by 2 days.

### 8.3 Definition of Done

The sprint is complete when all of the following are true:

- [object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object],[object Object]

## 9. File-by-File Change List & Code Snippets

### 9.1 New Files

The following files are created by this refactor. Each file has a single, well-defined purpose; no file exceeds ~300 lines. The new src/lib/brand/ directory is the home for all brand-resolution logic; the new src/app/admin/brand/ directory is the home for the admin UI; the new src/components/brand/brand-logo-server.tsx replaces the deprecated aisalon-logo-server.tsx.

- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]

- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]

## 9.2 Modified Files

The table below lists every modified file, the nature of the change, and an estimated line-count delta. Files are grouped by category. The largest change is the metadata refactor (~150 files, but each file is a 5-line change applied by the codemod, so the manual review burden is low — the codemod is deterministic and the diff is reviewable in a single PR).

**Table 5: Modified files (grouped by category)**

| Category | File | Change | Est. Lines |
|---|---|---|---|
| Schema | prisma/schema.prisma | Add Brand model + enum + FKs on User/Country | +80 |
| Schema | prisma/ schema.prisma.bak | Update header comment to reflect multi-brand | +1 |
| Middleware | src/middleware.ts | Add brand host-dispatch (gated by feature flag) | +25 |
| Email | src/lib/email.ts | Replace 4 signatures + from-address fallback with brand resolver | -30 +40 |
| Email | src/lib/email-orchestrator/ templates.ts | Tokenize SHELL + buildLogoBlock + 5 stage templates | -25 +60 |
| Email | src/lib/email-orchestrator/seed.ts | Update seed to use new tokens | -15 +15 |
| Email | src/lib/email/render-unified.ts | Add {{brand_*}} tokens + brand-aware unsubscribe footer | +30 |
| Email | src/lib/email-campaign/render.ts | Pass brand context through to renderUnifiedEmail | +5 |

| Category | File | Change | Est. Lines |
|---|---|---|---|
| Brand lib | src/lib/site-settings.ts | Add deprecation comments; keep functions for backward-compat | +10 |
| Brand lib | src/lib/chapter-brand-images.ts | Extend getEffectiveBrandImages to accept brand param; add brand.override.* lookup | +25 |
| Brand lib | src/lib/global-brand-library.ts | Add comment that this is now a fallback layer under Brand | +5 |
| Login | src/app/login/page.tsx | Replace hardcoded defaults with getBrandForRequest() (login agent PR) | -20 +30 |
| Login | src/app/login/login-form.tsx | Accept brand prop; use brand.name in copy (login agent PR) | +10 |
| Layout | src/app/layout.tsx | Add BrandProvider; replace hardcoded title template with generateMetadata | +20 |
| Admin | src/components/ais/admin-tabs-def.ts | Add Brand tab entry (SUPER_ADMIN only) | +8 |
| Admin | src/app/admin/chapters/page.tsx | Move GlobalBrandImagesEditor out; add redirect banner | -15 +5 |
| Logo | src/components/brand/aisalon-logo-server.tsx | Add @deprecated JSDoc; re-export BrandLogoServer for compat | +5 |
| Logo | src/components/brand/aisalon-logo.tsx | Same deprecation treatment | +5 |
| Metadata | ~150 files: src/app/**/page.tsx | Codemod: replace metadata = {title:'X — AI Salon'} with generateMetadata() | ~5 each |

| Category | File | Change | Est. Lines |
|---|---|---|---|
| Mockups | src/app/admin/mockups/*/sample-data.ts (5 files) | Replace 'AI Salon Tel Aviv' with brand-derived values | ~10 each |
| Mockups | src/app/admin/mockups/mockups-client.tsx | Pass brand context to mockup renderers | +15 |
| Config | .env.example | Add BRAND_DEFAULT_SLUG, BRAND_DOMAIN_RESOLVE_ENABLED, BRAND_HOST_MAP | +15 |
| Config | next.config.ts | Add explicit coma.massapro.com to images.remotePatterns | +5 |
| Tracking | src/lib/utm.ts | Keep 'AI Salon' UTM source for backward-compat; add comment | +3 |
| Seed | src/app/api/admin/v7-seed/route.ts | Insert Brand rows + backfill brandId on existing User/Country | +30 |

## 9.3 Representative Code Snippets

The following snippets are the canonical implementations referenced throughout this document. They are the contract between the platform agent and the login agent, and the contract between the platform agent and the email rebrand. Any deviation from these snippets must be agreed in writing.

### Snippet A — Prisma Brand model

(See Section 3.1 for the full model definition — repeated here as the canonical reference.)

```
// prisma/schema.prisma — addition
model Brand {
  id                String   @id @default(cuid())
  slug              String   @unique
  name              String
  wordmark          String
  tagline           String?
  domain            String   @unique
  altDomains        String[]
  siteUrl           String
  defaultChapterSlug  String?
```

```
  fromName            String
  fromEmail           String
  replyTo             String?
  supportEmail        String?
  logoUrl             String?
  logoDarkUrl         String?
  faviconUrl          String?
  emailLogoUrl        String?
  loginHeroUrl        String?
  loginBannerUrl      String?
  primaryColor        String?
  secondaryColor      String?
  accentColor         String?
  gradientCss         String?
  parentLogoUrl       String?
  parentLogoPosition  ParentLogoPosition @default(ABOVE)
  parentBrandName     String?
  parentBrandUrl      String?
  isActive            Boolean  @default(true)
  sortOrder           Int      @default(0)
  createdAt           DateTime @default(now())
  updatedAt           DateTime @updatedAt
  countries           Country[]
  users               User[]
  @@map("brand")
}
enum ParentLogoPosition { ABOVE; BESIDE; FOOTER_ONLY; HIDDEN }
```

### Snippet B — getEffectiveBrand(context) resolver

(See Section 3.3 for the full implementation.)

```
// src/lib/brand/resolver.ts (excerpt)
export async function getEffectiveBrand(
  ctx: { chapterId?: string; countryId?: string; host?: string } = {}
): Promise<BrandContext> {
  // 1. chapter → 2. country → 3. host → 4. env default
  // (full body in Section 3.3)
}
```

### Snippet C — Middleware host-dispatch diff

(See Section 4.1 for the full diff.)

```
// src/middleware.ts — additions only
if (process.env.BRAND_DOMAIN_RESOLVE_ENABLED === 'true') {
  const host = req.headers.get('x-forwarded-host') ||
               req.headers.get('host') || '';
  const slug = resolveBrandSlugByHost(host);
  if (slug) {
    res.headers.set('x-resolved-brand-slug', slug);
    req.headers.set('x-resolved-brand-slug', slug);
  }
}
```

### Snippet D — buildLogoBlock with parent coexistence

(See Section 6.4 for the full implementation — handles ABOVE / BESIDE / FOOTER_ONLY / HIDDEN.)

```
// src/lib/email-orchestrator/templates.ts (excerpt)
function buildLogoBlock(brand: Brand, parent: Brand | null): string {
  // Returns two-row table (parent above, smaller) when brand.parentLogoPosition ===
'ABOVE'
  // Returns single logo when no parent or HIDDEN
  // (full body in Section 6.4)
}
```

### Snippet E — Email from-address resolution

(See Section 6.5.)

```
// src/lib/email.ts (excerpt)
async function resolveFromAddress(brand: Brand): Promise<string> {
  if (process.env.SMTP_FROM) return process.env.SMTP_FROM;
  return `${brand.fromName} <${brand.fromEmail}>`;
}
```

### Snippet F — Login page.tsx top section

(See Section 5.4 — owned by the login agent.)

```
// src/app/login/page.tsx (excerpt)
export default async function LoginPage({ searchParams }) {
  const { brand, parent } = await getBrandForRequest();
  const chapterSlug = searchParams.chapterSlug || brand.defaultChapterSlug;
  // ... passes brand + parent to BrandLogoServer
}
```

### Snippet G — BrandLogoServer component

(See Section 5.5 — owned by the platform agent.)

```
// src/components/brand/brand-logo-server.tsx (excerpt)
export function BrandLogoServer({ brand, parent, variant, markSrc }: Props) {
  // Renders parent.logoUrl above brand.logoUrl when brand.parentLogoPosition ===
'ABOVE'
  // (full body in Section 5.5)
}
```

### Snippet H — generateMetadata pattern for admin pages

This is the codemod target for the ~150 admin/page.tsx files. The pattern replaces the static `metadata` export with a `generateMetadata` async function that reads the brand. The codemod is deterministic — it matches the exact string `title: "X — AI Salon` and replaces it with the template below, preserving the page-specific prefix ("X").

```
// Codemod target pattern (before):
```

```
export const metadata = {
  title: "Members — AI Salon",
  description: "Manage members of AI Salon",
};

// Codemod output (after):
import { getBrandForRequest } from "@/lib/brand/resolver";
export async function generateMetadata() {
  const { brand } = await getBrandForRequest();
  return {
    title: `Members — ${brand.name}`,
    description: `Manage members of ${brand.name}`,
  };
}
```

# 10. SQL Migration & Backfill

## 10.1 Migration SQL

The migration is generated by `npx prisma migrate dev --name add_brand_model` and committed to `prisma/migrations/{timestamp}_add_brand_model/migration.sql`. The SQL below is the canonical version; the actual migration file may differ slightly in formatting (Prisma's generator uses uppercase keywords and specific indentation). The migration is run against the production database as part of PR#1's deployment.

```
-- prisma/migrations/{timestamp}_add_brand_model/migration.sql

-- Step 1: Create ParentLogoPosition enum
CREATE TYPE "ParentLogoPosition" AS ENUM ('ABOVE', 'BESIDE', 'FOOTER_ONLY',
'HIDDEN');

-- Step 2: Create brand table
CREATE TABLE "brand" (
    "id"                TEXT NOT NULL,
    "slug"              TEXT NOT NULL,
    "name"              TEXT NOT NULL,
    "wordmark"          TEXT NOT NULL,
    "tagline"           TEXT,
    "description"       TEXT,
    "domain"            TEXT NOT NULL,
    "altDomains"        TEXT[] DEFAULT ARRAY[]::TEXT[],
    "siteUrl"           TEXT NOT NULL,
    "defaultChapterSlug" TEXT,
    "supportEmail"      TEXT,
    "fromName"          TEXT NOT NULL,
    "fromEmail"         TEXT NOT NULL,
    "replyTo"           TEXT,
    "logoUrl"           TEXT,
    "logoDarkUrl"       TEXT,
    "faviconUrl"        TEXT,
```

```
    "emailLogoUrl"       TEXT,
    "loginHeroUrl"       TEXT,
    "loginBannerUrl"     TEXT,
    "primaryColor"       TEXT,
    "secondaryColor"     TEXT,
    "accentColor"        TEXT,
    "gradientCss"        TEXT,
    "parentLogoUrl"      TEXT,
    "parentLogoPosition" "ParentLogoPosition" NOT NULL DEFAULT 'ABOVE',
    "parentBrandName"    TEXT,
    "parentBrandUrl"     TEXT,
    "isActive"           BOOLEAN NOT NULL DEFAULT true,
    "sortOrder"          INTEGER NOT NULL DEFAULT 0,
    "createdAt"          TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt"          TIMESTAMP(3) NOT NULL,

    CONSTRAINT "brand_pkey" PRIMARY KEY ("id")
);

CREATE UNIQUE INDEX "brand_slug_key" ON "brand" ("slug");
CREATE UNIQUE INDEX "brand_domain_key" ON "brand" ("domain");

-- Step 3: Add nullable brandId to user + country (chapter inherits via country)
ALTER TABLE "user"    ADD COLUMN "brandId" TEXT;
ALTER TABLE "country" ADD COLUMN "brandId" TEXT;

-- Step 4: Foreign keys + indexes
ALTER TABLE "user"    ADD CONSTRAINT "user_brandId_fkey"
  FOREIGN KEY ("brandId") REFERENCES "brand"("id") ON DELETE SET NULL ON
UPDATE CASCADE;
ALTER TABLE "country" ADD CONSTRAINT "country_brandId_fkey"
  FOREIGN KEY ("brandId") REFERENCES "brand"("id") ON DELETE SET NULL ON
UPDATE CASCADE;
CREATE INDEX "user_brandId_idx"    ON "user"("brandId");
CREATE INDEX "country_brandId_idx" ON "country"("brandId");
```

## 10.2 Backfill SQL

The backfill is run inside the same migration, in a transaction. If any step fails, the entire migration rolls back. The backfill inserts two Brand rows (Coma first, AI Salon second), then updates every existing User and Country row to point at the AI Salon brand. Estimated row counts (to size the backfill window): Users ~5,000, Countries ~20, Chapters ~50. The backfill should complete in under 5 seconds on a production-grade Postgres instance.

```
-- Step 5: Insert Coma brand (parent — no parent of its own)
INSERT INTO "brand" (
  "id", "slug", "name", "wordmark", "tagline",
  "domain", "siteUrl",
  "fromName", "fromEmail", "supportEmail",
  "logoUrl", "faviconUrl", "emailLogoUrl",
  "primaryColor", "secondaryColor", "accentColor", "gradientCss",
  "parentLogoPosition", "isActive", "sortOrder", "createdAt", "updatedAt"
) VALUES (
  'brand_coma', 'coma', 'Coma', 'coma', 'The platform behind AI Salon',
```

```
  'coma.massapro.com', 'https://coma.massapro.com',
  'Coma', 'no-reply@coma.massapro.com', 'support@coma.massapro.com',
  'https://coma.massapro.com/assets/coma_trans.png',
  'https://coma.massapro.com/assets/coma_favicon.png',
  'https://coma.massapro.com/assets/coma_trans.png',
  '#0F172A', '#475569', '#0042E6',
  'linear-gradient(135deg, #0F172A 0%, #475569 100%)',
  'HIDDEN', true, 0, NOW(), NOW()
);

-- Step 6: Insert AI Salon brand (child — references Coma as parent)
INSERT INTO "brand" (
  "id", "slug", "name", "wordmark", "tagline",
  "domain", "siteUrl", "defaultChapterSlug",
  "fromName", "fromEmail", "supportEmail",
  "logoUrl", "faviconUrl", "emailLogoUrl", "loginHeroUrl", "loginBannerUrl",
  "primaryColor", "secondaryColor", "accentColor", "gradientCss",
  "parentLogoUrl", "parentLogoPosition", "parentBrandName", "parentBrandUrl",
  "isActive", "sortOrder", "createdAt", "updatedAt"
) VALUES (
  'brand_aisalon', 'aisalon', 'AI Salon', 'aisalon', 'Empowering AI Connections',
  'aisalon.massapro.com', 'https://aisalon.massapro.com', 'tel-aviv',
  'AI Salon Chat', 'chat@aisalon.massapro.com', 'support@aisalon.massapro.com',
  'https://aisalon.massapro.com/brand/aisalon-logo.webp',
  'https://aisalon.massapro.com/favicon.webp',
  'https://aisalon.massapro.com/brand/aisalon-logo.webp',
  'https://aisalon.massapro.com/images/falafel-meerkat.jpg',
  'https://aisalon.massapro.com/brand/aisalon-logo.webp',
  '#FF005A', '#00E6FF', '#820A7D',
  'conic-gradient(from 180deg at 50% 50%, #FF005A, #820A7D, #004F98, #00E6FF,
#FF005A)',
  'https://coma.massapro.com/assets/coma_trans.png', 'ABOVE', 'Coma',
'https://coma.massapro.com',
  true, 1, NOW(), NOW()
);

-- Step 7: Insert example external client brand (google — for sales demos)
-- This row is what makes https://coma.massapro.com/login?
brand=google&chapterSlug=tlv
-- render the Google logo + Coma parent above. ANY future client brand works the same
-- way: insert a row with their logoUrl, parentBrandId='brand_coma',
parentLogoPosition='ABOVE'.
INSERT INTO "brand" (
  "id", "slug", "name", "wordmark", "tagline",
  "domain", "siteUrl", "defaultChapterSlug",
  "fromName", "fromEmail", "supportEmail",
  "logoUrl", "faviconUrl", "emailLogoUrl", "loginHeroUrl", "loginBannerUrl",
  "primaryColor", "secondaryColor", "accentColor", "gradientCss",
  "parentLogoUrl", "parentLogoPosition", "parentBrandName", "parentBrandUrl",
  "isActive", "sortOrder", "createdAt", "updatedAt"
) VALUES (
  'brand_google', 'google', 'Google', 'google', 'Organize the world's information',
  'google.com', 'https://www.google.com', NULL,
  'Google Workspace', 'no-reply@google.com', 'support@google.com',

'https://www.google.com/images/branding/googlelogo/2x/googlelogo_color_272x88dp.png',
  'https://www.google.com/favicon.ico',

'https://www.google.com/images/branding/googlelogo/2x/googlelogo_color_272x88dp.png',
```

```
  NULL, NULL,
  '#4285F4', '#34A853', '#EA4335',
  'linear-gradient(135deg, #4285F4 0%, #34A853 100%)',
  'https://coma.massapro.com/assets/coma_trans.png', 'ABOVE', 'Coma',
'https://coma.massapro.com',
  true, 2, NOW(), NOW()
);

-- Step 8: Backfill FKs on existing rows
UPDATE "user"    SET "brandId" = 'brand_aisalon' WHERE "brandId" IS NULL;
UPDATE "country" SET "brandId" = 'brand_aisalon' WHERE "brandId" IS NULL;
-- Chapter inherits via Country — no chapter.brandId column.
```

The google seed row is intentionally a demo-only entry — it has `defaultChapterSlug = NULL` (Google is not chapter-based) and `siteUrl = 'https://www.google.com'` (the brand's external home, not a massapro.com subdomain). When a request comes in with `?brand=google&chapterSlug=tlv`, the resolver returns the google Brand row, and the login page renders the Google logo + Coma parent above + "Tel Aviv" as the chapter name in the title (because chapterSlug=tlv is in the URL). This is the pattern for ANY future client brand — one INSERT, zero code change.

## 10.3 Post-Backfill Verification + NOT NULL

After the backfill, a verification query confirms every row has a `brandId`. If any row is null (indicating a concurrent insert during the migration window), the verification fails and the NOT NULL constraint is NOT added. A follow-up migration (run after the verification passes) adds the NOT NULL constraint. This two-step approach avoids a single-migration failure mode where the NOT NULL constraint would fail if any row was missed.

```
-- Verification query (run manually after migration, before follow-up)
SELECT
  (SELECT COUNT(*) FROM "user"    WHERE "brandId" IS NULL) AS null_users,
  (SELECT COUNT(*) FROM "country" WHERE "brandId" IS NULL) AS null_countries;
-- Expected: both 0. If non-zero, investigate + manually backfill the missing rows.

-- Follow-up migration (run only after verification passes):
ALTER TABLE "user"    ALTER COLUMN "brandId" SET NOT NULL;
ALTER TABLE "country" ALTER COLUMN "brandId" SET NOT NULL;
```

## 10.4 Rollback SQL

The rollback is the inverse of the migration. It must be run only after the feature flag `BRAND_DOMAIN_RESOLVE_ENABLED` has been set to false and the code that reads `Brand` has been reverted. Running the rollback while the code still reads `Brand` will cause runtime errors (every `getEffectiveBrand()` call will throw).

```
-- Rollback migration
ALTER TABLE "user"    DROP CONSTRAINT "user_brandId_fkey";
```

```
ALTER TABLE "country" DROP CONSTRAINT "country_brandId_fkey";
DROP INDEX "user_brandId_idx";
DROP INDEX "country_brandId_idx";
ALTER TABLE "user"    DROP COLUMN "brandId";
ALTER TABLE "country" DROP COLUMN "brandId";
DROP TABLE "brand";
DROP TYPE "ParentLogoPosition";
```

### 10.5 Migration Safety

The migration is wrapped in a transaction (Prisma's default behavior). If any statement fails, the entire migration rolls back and the database is unchanged. The backfill UPDATEs are idempotent — they only affect rows where `brandId IS NULL`, so a partial failure + retry does not double-apply. The INSERTs use hardcoded IDs (`brand_coma`, `brand_aisalon`) so re-running the migration on a partially-migrated database produces a primary-key conflict on the second run, which is the correct behavior (the migration has already been applied).

Estimated migration duration on a production Postgres instance: under 10 seconds total (CREATE TABLE is instant; the two INSERTs are single-row; the two UPDATEs scan ~5,000 + 20 rows). The migration can be run during normal traffic — no maintenance window is required. The application continues to serve requests throughout the migration because the new columns are nullable and the existing code (pre-PR#2) does not read them.

## 11. Test Plan

### 11.1 Unit Tests

Unit tests live alongside the source files (`*.test.ts` pattern) and run in CI on every PR. The tests below use Vitest (the existing test runner). Each test is independent and uses a mocked Prisma client — no real database access. The brand resolver tests cover the fallback chain (chapter → country → host → env default) and the parent-brand resolution.

1. resolveBrandSlugByHost returns 'coma' for coma.massapro.com (explicit map).
2. resolveBrandSlugByHost returns 'aisalon' for aisalon.massapro.com (explicit map).
3. resolveBrandSlugByHost returns 'aisalon' for nyc.aisalon.massapro.com (subdomain heuristic).
4. resolveBrandSlugByHost returns BRAND_DEFAULT_SLUG env value for localhost.
5. resolveBrandSlugByHost returns null for an unknown host (no env default fallback in this function).
6. resolveBrandSlugByHost strips port from hostname (localhost:3000 → localhost).
7. getEffectiveBrand returns chapter brand when ctx.chapterId resolves to a chapter with country.brand.

8. getEffectiveBrand falls through to country brand when ctx.chapterId has no country.brand.
9. getEffectiveBrand falls through to host brand when chapter + country have no brand.
10. getEffectiveBrand falls through to env default when host is unknown.
11. getEffectiveBrand throws when env default slug is not in DB (deployment misconfiguration).
12. getEffectiveBrand returns parent brand when brand.parentLogoUrl + parentBrandName are set.
13. getEffectiveBrand returns parent=null when brand.parentLogoUrl is null (Coma brand itself).
14. buildLogoBlock returns single-logo HTML when parent is null.
15. buildLogoBlock returns two-row table HTML when parentLogoPosition is ABOVE.
16. buildLogoBlock returns two-column table HTML when parentLogoPosition is BESIDE.
17. buildLogoBlock returns single-logo HTML when parentLogoPosition is FOOTER_ONLY (parent in signature, not header).
18. buildLogoBlock returns single-logo HTML when parentLogoPosition is HIDDEN.
19. renderUnifiedEmail replaces {{brand_name}} with brand.name.
20. renderUnifiedEmail replaces {{parent_brand_name}} with parent.name when parent exists.
21. renderUnifiedEmail replaces {{parent_brand_name}} with empty string when parent is null.
22. renderUnifiedEmail replaces {{brand_url}} with brand.siteUrl.
23. renderUnifiedEmail leaves template-specific tokens (e.g. {{speakerName}}) untouched.
24. resolveFromAddress returns SMTP_FROM env value when set (operator override).
25. resolveFromAddress returns 'Brand Name <from@brand.com>' when SMTP_FROM is unset.

## 11.2 Integration Tests

Integration tests run against a real Postgres instance (via Docker Compose in CI) and exercise the full request → middleware → resolver → render path. They use a separate test database seeded with the two Brand rows + a test chapter + a test user. Each test rolls back its DB changes in a transaction to keep tests isolated.

1. GET /login on coma.massapro.com (host header set) returns HTML containing 'Coma' in the <title> and the Coma logo URL.
2. GET /login on aisalon.massapro.com (host header set) returns HTML containing 'AI Salon' in the <title> and the AI Salon logo URL.
3. GET /login on coma.massapro.com does NOT contain the string 'AI Salon' anywhere in the rendered HTML.

4. POST /api/auth/signup on coma.massapro.com creates a User row with brandId='brand_coma'.
5. POST /api/auth/signup on aisalon.massapro.com creates a User row with brandId='brand_aisalon'.
6. GET /admin/brand returns 200 for SUPER_ADMIN session.
7. GET /admin/brand returns 403 for ADMIN session (not elevated enough).
8. GET /admin/brand returns 403 for CHAPTER_ORGANIZER session.
9. GET /admin/brand returns 302 to /login for unauthenticated request.
10. POST /api/admin/brand with valid payload creates a new Brand row in the DB.
11. POST /api/admin/brand with duplicate slug returns 409 Conflict.
12. POST /api/admin/brand with missing required fields (name, slug, domain) returns 400.
13. PATCH /api/admin/brand/[id] with isActive=false sets the brand inactive; subsequent GET /login on that brand's host returns 404.
14. POST /api/admin/brand/[id]/override with {chapterId, field:'logoUrl', value:'https://...'} creates a ChapterSetting row with key 'brand.override.logoUrl'.
15. DELETE /api/admin/brand/[id]/override?chapterId=...&field=logoUrl removes the ChapterSetting row.

## 11.3 End-to-End Tests (Playwright)

E2E tests run against a full deployment (staging environment) using Playwright. They cover the full user journey across both brands. Visual email regression uses a mailcatcher instance in CI — emails are sent to the mailcatcher, then fetched via its API and the HTML is diffed against a snapshot. Snapshot drift fails the test.

1. E2E-1: Full signup flow on coma.massapro.com — visit /login, fill signup form, submit, verify email received with Coma branding, click verify link, verify user is logged in and sees Coma brand in the app header.
2. E2E-2: Full signup flow on aisalon.massapro.com — same as E2E-1 but with AI Salon branding + Coma parent logo in email header.
3. E2E-3: Login as existing AI Salon user on aisalon.massapro.com — verify user sees AI Salon brand as before the refactor (no regression).
4. E2E-4: Login as existing AI Salon user on coma.massapro.com — verify user is told their account does not exist on this brand (expected behavior, since brandId is set at signup).
5. E2E-5: Admin brand create-edit-delete flow — login as SUPER_ADMIN, visit /admin/brand, create a test brand, edit its fields, deactivate it, verify it disappears from the active list.
6. E2E-6: Email visual regression — trigger a password-reset email on both brands, fetch the HTML from mailcatcher, diff against snapshot. Snapshot must show correct logo + parent logo (for AI Salon) + signature + from-address.

7. E2E-7: Email visual regression — trigger an orchestrator RSVP-confirmation email on both brands, same snapshot diff.

## 11.4 Manual Smoke Test Checklist

Before enabling the feature flag in production, run this checklist manually against the staging deployment. Each item is a single click or curl command. The checklist takes ~30 minutes to complete. The platform agent owns items 1-7; the login agent owns items 8-10; the SUPER_ADMIN on call owns items 11-15.

1. Visit https://aisalon.massapro.com/login — verify AI Salon logo + AIS colors + 'Login — AI Salon Tel Aviv' title.
2. Visit https://coma.massapro.com/login — verify Coma logo + Coma colors + 'Login — Coma' title.
3. curl -s https://coma.massapro.com/login | grep -c 'AI Salon' — expect 0 (no AI Salon strings in Coma HTML).
4. curl -s https://aisalon.massapro.com/login | grep -c 'AI Salon' — expect >0 (AI Salon strings present in AI Salon HTML).
5. Trigger a password-reset email for a test user on aisalon.massapro.com — verify the email has AI Salon logo + Coma parent logo above + AI Salon signature with Coma credit.
6. Trigger a password-reset email for a test user on coma.massapro.com — verify the email has Coma logo only (no parent) + Coma signature.
7. Verify the unsubscribe link in the AI Salon email points to https://aisalon.massapro.com/api/email/unsubscribe?token=...
8. Verify the unsubscribe link in the Coma email points to https://coma.massapro.com/api/email/unsubscribe?token=...
9. Visit https://aisalon.massapro.com/favicon.ico — verify it's the AI Salon favicon (visual check in browser tab).
10. Visit https://coma.massapro.com/favicon.ico — verify it's the Coma favicon.
11. Visit /admin/brand as SUPER_ADMIN — verify the Brands tab loads with 2 brands (Coma + AI Salon).
12. Visit /admin/brand as ADMIN — verify 403 Forbidden.
13. Click 'New Brand' in /admin/brand — fill in a test brand 'Acme' with domain 'acme.test' — verify it saves and appears in the list.
14. Toggle BRAND_DOMAIN_RESOLVE_ENABLED=false in Vercel env vars, redeploy, visit https://coma.massapro.com/login — verify it falls back to AI Salon brand (rollback path).
15. Re-toggle BRAND_DOMAIN_RESOLVE_ENABLED=true, redeploy, verify Coma brand is back on coma.massapro.com.

### 11.5 Performance

The middleware host-dispatch adds a single hash-map lookup per request — the `BRAND_HOST_MAP` env var is parsed once at module load and cached. The expected per-request latency overhead is under 1ms. A k6 load test (1,000 virtual users, 60 seconds, mixed traffic across both hosts) should show no statistically significant latency increase vs. the pre-refactor baseline. The Prisma query in `getEffectiveBrand()` is a single primary-key lookup on the Brand table (indexed by slug), expected to complete in under 5ms — acceptable for per-request invocation. If profiling shows the resolver is a hotspot, it can be wrapped in React's `cache()` to deduplicate within a single request.

## 12. Risks & Rollback Plan

### 12.1 Risk Register

The risk register below is the canonical list reviewed at sprint kickoff and at every PR merge. Each risk has a likelihood (Low/Med/High), impact (Low/Med/High), mitigation, and owner. Risks marked with a likelihood of Med or higher are tracked in the sprint retro.

**Table 6: Risk register**

| Risk | Lkhd | Impact | Mitigation | Owner |
|---|---|---|---|---|
| Middleware host-detection breaks on Vercel edge (x-forwarded-host not set) | Med | High | Feature flag BRAND_DOMAIN_RESOLVE_ENABLED; fallback to BRAND_DEFAULT_SLUG on unknown host; tested in staging before prod. | Platform |
| Backfill SQL fails on large Users table (5K+ rows) | Low | High | Run in transaction; gate with verification query before NOT NULL; manual backfill script for any null rows. | Platform |

| Risk | Lkhd | Impact | Mitigation | Owner |
|---|---|---|---|---|
| 150-file metadata codemod introduces typos or breaks page titles | Med | Med | Automated codemod with deterministic patterns; visual smoke test of 10 sample admin pages; full diff review in single PR. | Platform |
| Parallel agent's login PR (PR#5) conflicts with platform's logo refactor (PR#7) | Med | Med | Agree on BrandLogoServer interface upfront (this doc, Section 5.5); merge PR#5 before PR#7 starts; PR#7 then mechanical. | Both |
| Email token replacer breaks existing templates (missing token → literal {{brand_name}} in email body) | Low | High | Visual email regression tests for both brands; mailcatcher in CI; manual smoke test of all 5 stage templates. | Platform |
| Coma parent logo on AI Salon emails perceived as 'Coma owns AI Salon' — brand perception risk | Med | Low | Per-brand parentLogoPosition setting (FOOTER_ONLY available); chapter-level override possible; review with chapter leads before prod. | Product |
| DNS misconfiguration on coma.massapro.com takes both brands down (shared Vercel deployment) | Low | High | DNS change made first, verified, then env var + feature flag enabled; instant rollback via feature flag. | Platform |

| Risk | Lkhd | Impact | Mitigation | Owner |
|---|---|---|---|---|
| Existing users see unexpected Coma logo on AI Salon pages (perception of change) | Med | Low | Coma logo is small (80px) and above the AI Salon logo (150px); communicate change to chapter leads 1 week before; provide FOOTER_ONLY override for sensitive chapters. | Product |
| Seed migration script (migrate-email-templates-to-brand-tokens.ts) fails on production DB | Low | High | Run against staging DB first; idempotent (safe to re-run); rollback by re-seeding with old hardcoded templates via existing v7-seed endpoint. | Platform |
| New Brand admin UI (PR#4) has permission bypass — non-SUPER_ADMIN can create brands | Low | High | Reuse existing permission check pattern from /api/admin/*; integration test for 403 on ADMIN/CHAPTER_ORGANIZER; security review before merge. | Platform |
| Feature flag default-off in production means DNS-configured coma.massapro.com shows AI Salon brand until flag is toggled | Med | Low | Document the flag-toggle step in PR#9 deploy runbook; SUPER_ADMIN on call toggles flag after smoke test. | Platform |

## 12.2 Rollback Plan

The rollback plan is layered — we can roll back at four levels of granularity, from instant (feature flag) to slow (schema revert). Each level is independently executable; the level chosen depends on

the severity of the issue. The feature flag is the primary rollback mechanism and should be the first action in any incident.

### Level 1 — Feature flag (instant, <1 minute)

Set BRAND_DOMAIN_RESOLVE_ENABLED=false in Vercel env vars and trigger a redeploy. All requests fall back to BRAND_DEFAULT_SLUG (AI Salon). Visiting coma.massapro.com shows the AI Salon brand. The Brand table, the resolver, the email rebrand — all remain in place, but they are bypassed at the edge. This is the rollback for any brand-detection issue, any wrong-brand-rendering issue, or any Coma-brand-specific outage. Cost: 1 env-var change + 1 redeploy (~2 minutes on Vercel).

### Level 2 — Revert middleware PR (5-10 minutes)

If the feature flag is insufficient (e.g., the middleware is causing request hangs), revert PR#2 (the middleware changes). The middleware returns to UTM-tracking-only behavior. The Brand table, resolver, and email rebrand remain in place but are not invoked. This is the rollback for middleware-specific issues. Cost: 1 revert PR + 1 deploy.

### Level 3 — Revert email rebrand PR (15-30 minutes)

If emails are rendering incorrectly (broken tokens, missing logos), revert PR#3 (the email rebrand). Re-run the v7-seed endpoint to restore the old hardcoded template strings. The Brand table, resolver, and middleware remain in place but emails revert to AI Salon hardcoded branding. This is the rollback for email-specific issues. Cost: 1 revert PR + 1 seed re-run + 1 deploy.

### Level 4 — Schema revert (30-60 minutes, only after Level 1)

If the Brand table itself is causing issues (e.g., a bad migration left the DB in an inconsistent state), run the rollback SQL from Section 10.4. This MUST be done only after Level 1 (feature flag off) and after reverting PR#2 (so no code reads the Brand table). Running the schema revert while the code still reads Brand will cause runtime errors. Cost: 1 SQL migration + verification.

## 12.3 Communication Plan

One week before the production cut-over (PR#9), notify all chapter leads via email. The notification explains: (1) the Coma parent logo will appear above the AI Salon logo on web pages and emails, (2) the change is reversible per-chapter via the Chapter Overrides tab, (3) the cut-over date and the expected 5-minute window of brand-flicker as the feature flag propagates. A status page update goes live on both coma.massapro.com and aisalon.massapro.com 24 hours before the cut-over. The rollback decision owner is the SUPER_ADMIN on call — they have the authority to toggle the feature flag without further approval if any of the smoke-test items fail.

### 12.4 Post-Migration Monitoring

For 7 days after the cut-over, monitor the following metrics. Anomalies trigger an investigation but not an automatic rollback (rollback is a human decision).

- [object Object],[object Object],[object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]
- [object Object],[object Object]

## 13. Appendix A — Inventory of "AI Salon" Brand Surfaces

This appendix is the working checklist for the metadata refactor (PR#6) and the email rebrand (PR#3). It is derived from the codebase exploration performed before writing this plan (565 occurrences of the pattern AI Salon | AISalon | ai-salon | aisalon across 185 files in /src, plus 3 files in /prisma and 8 in /public). The table groups files by category and assigns each a refactor action. Files marked KEEP are intentionally not refactored — they are either backward-compat shims, historical seed data, or UTM campaign identifiers that must remain stable for analytics continuity.

**Table 7: Inventory of "AI Salon" brand surfaces (565 occurrences, 185 files)**

| Category | Files (count) / Occurrences | Refactor Action |
|---|---|---|
| UI strings — admin page metadata | ~150 files (src/app/**/page.tsx), 1 occ each | Codemod PR#6: replace metadata = {title:'X — AI Salon'} with generateMetadata() reading brand.name |
| Email templates — transactional | src/lib/email.ts (28 occ) | PR#3: replace 4 signatures + from-address fallback with brand resolver + tokens |
| Email templates — orchestrator SHELL | src/lib/email-orchestrator/templates.ts (18 occ) | PR#3: tokenize SHELL + buildLogoBlock + 5 stage templates with {{brand_*}} tokens |
| Email templates — seed | src/lib/email-orchestrator/seed.ts (5 occ) | PR#3: update seed to use new tokens; run migrate-email-templates script on prod DB |

| Category | Files (count) / Occurrences | Refactor Action |
|---|---|---|
| Email renderer | src/lib/email/render-unified.ts (2 occ) | PR#3: replace 'AI Salon' in unsubscribe footer with {{brand_name}} token |
| Email campaign renderer | src/lib/email-campaign/render.ts (2 occ) | PR#3: pass brand context through to renderUnifiedEmail |
| Email APIs — campaigns | src/app/api/admin/email/campaigns/[id]/{send,test-send}/route.ts (9 occ) | PR#3: read brand from getEffectiveBrand() and pass to render |
| Email APIs — unsubscribe + cron | src/app/api/email/unsubscribe/route.ts (5 occ), src/app/api/cron/email/route.ts (5 occ) | PR#3: brand-aware unsubscribe URL; cron reads brand per campaign |
| Email APIs — DM notifications | src/app/api/messages/[userId]/route.ts (7 occ) | PR#3: use brand.name in DM notification body + signature |
| Email APIs — speaker messages | src/app/api/speakers/[id]/messages/route.ts (4 occ) | PR#3: use brand.name in speaker message body |
| Login page | src/app/login/page.tsx (13 occ) | PR#5 (login agent): full brand-swap per Section 5 |
| Layout metadata | src/app/layout.tsx (9 occ) | PR#6: replace title template + OG metadata with generateMetadata reading brand |
| Chapter onboarding form | src/app/chapter-onboarding/[token]/chapter-onboarding-form.tsx (12 occ), page.tsx (10 occ) | PR#6: brand-aware copy; onboarding is AI-Salon-only for now (Coma is not chapter-based) |
| Admin — knowledge base | src/app/admin/knowledge-base/page.tsx (14 occ) | PR#6: metadata refactor + brand-aware page header |
| Admin — mockups | src/app/admin/mockups/mockups-client.tsx (10 occ), sample-data.ts (5 files, 30 occ total) | PR#8: brand-derived event titles + speaker bios; remove hardcoded 'AI Salon Tel Aviv' |
| Onboarding | src/app/onboarding/page.tsx (8 occ) | PR#6: metadata refactor + brand-aware onboarding copy |
| Events page | src/app/events/page.tsx (8 occ) | PR#6: metadata refactor |
| Chapter landing | src/app/c/[chapterSlug]/chapter-landing-client.tsx (7 occ) | PR#6: brand-aware landing copy; pass brand from server component |

| Category | Files (count) / Occurrences | Refactor Action |
|---|---|---|
| Public event page | src/app/e/[slug]/public-event-page.tsx (7 occ) | PR#6: brand-aware event page metadata + copy |
| Admin email tab | src/app/admin/email/email-tab-client.tsx (7 occ) | PR#6: brand-aware admin UI strings |
| Referral share card | src/components/ais/referral-share-card.tsx (6 occ) | PR#7: brand-aware share-card text + image |
| Campaign composer | src/app/admin/email/campaign-composer.tsx (6 occ) | PR#6: brand-aware composer copy |
| Logo components | src/components/brand/aisalon-logo.tsx (5 occ), aisalon-logo-server.tsx (5 occ) | PR#7: deprecate (add @deprecated JSDoc, re-export BrandLogoServer); do NOT delete in this sprint |
| UTM tracking | src/lib/utm.ts (5 occ), src/lib/tracking/* (8 occ total) | KEEP: 'AI Salon' UTM source values must remain stable for existing campaign analytics continuity |
| Site settings lib | src/lib/site-settings.ts (5 occ) | PR#3: add deprecation comments; keep functions as fallback layer under Brand fields |
| Email orchestrator workers | src/lib/email-orchestrator/{worker,sender,flow-worker}.ts (8 occ total) | PR#3: pass brand context through to renderer |
| Env / config | .env.example (3 occ — SMTP_FROM examples), next.config.ts (1 occ — images.remotePatterns) | PR#2: update .env.example with BRAND_* vars; add coma.massapro.com to next.config.ts images.remotePatterns |
| Public assets | public/brand-book.md, public/email-system-doc.html, public/register-to-checkin-journey.html, public/previews/email-awareness-preview.html, public/ai-human-flourishing-booklet*.html (4 variants), public/human-flourishing-booklet-print.html | KEEP: static documentation; update in a future doc-refresh sprint |

| Category | Files (count) / Occurrences | Refactor Action |
|---|---|---|
| Prisma schema header | prisma/schema.prisma (1 occ — header comment), prisma/schema.prisma.bak, prisma/schema.sqlite-sandbox.prisma | PR#1: update header comment to reflect multi-brand platform |
| Prisma seed (HTTP-triggered) | src/app/api/admin/v7-seed/route.ts (3 occ) | PR#1: insert Brand rows + backfill brandId on existing User/Country; KEEP historical 'AI Salon' strings for re-seed compat |
| Brand image library | src/lib/global-brand-library.ts (1 occ) | KEEP: hardcoded AI Salon Blob URLs remain as final fallback layer |
| Brand image uploads | public/uploads/brand-assets/ (4 committed files) | KEEP: existing uploads; new brand uploads go to same directory per-brand |
| Chapter core defaults | public/defaults/chapter-core/ (5 files), src/lib/chapter-core.ts | KEEP: canonical AI Salon default brand pack for new chapters |
| Brand book + favicon | public/brand-book.md, public/brand/aisalon-logo.webp, public/favicon.ico, public/images/falafel-meerkat.{jpg,png} | KEEP: canonical AI Salon brand assets; Coma assets live at coma.massapro.com/assets/ |

After PR#6, PR#7, PR#8, and PR#3 merge, a verification grep should return only the KEEP categories: UTM tracking (8 occurrences in src/lib/utm.ts + src/lib/tracking/*), deprecated logo components (10 occurrences in src/components/brand/aisalon-logo*.tsx), historical seed data (3 occurrences in src/app/api/admin/v7-seed/route.ts), static public assets (~30 occurrences across /public HTML/PDF files), and the brand image library (1 occurrence in src/lib/global-brand-library.ts). Total expected post-refactor count: ~52 occurrences across ~15 files — a 91% reduction from the current 565/185.